PHÂN HIỆU TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TẠI TP. HỒ CHÍ MINH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

----ꕥ🕮ꕥ----



KHAI PHÁ DỮ LIỆU

BÁO CÁO THỰC HÀNH BUỔI 2

Nguyễn Đức Vinh - 6451071087

CQ.64.CNTT

GVHD: Ths. Nguyễn Thiện Dương

TP. Hồ Chí Minh - 2026

Lưu ý:

1. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
2. *SV bố trí mỗi câu ở dưới sẽ là 1 file mã nguồn riêng (file.ipynb)*
3. *Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó*
4. ***Hình chụp kết quả phải có chứa thanh taskbar của máy tính (có kèm theo thời gian và ngày làm bài)***
5. *SV đặt tên cho file mã nguồn mỗi câu như sau:* ***<MSSV>_LAB2_Bai<X>***
6. *Tất cả các câu sau đó sẽ tổng hợp lại và push lên 1 repository duy nhất trên Github*
7. *Đặt tên cho repository theo cú pháp:* ***<MSSV>_Lab2***

BÁO CÁO THỰC HÀNH

SV paste link dẫn đến 1 repo duy nhất của tất cả các câu bên dưới tại dây (lưu ý để ở chế độ public)

https://github.com/Drista4real/6451071087_Lab2.git

Bài 2 (Trang 15-16) – Adult Income

Câu 1: Nhập dữ liệu và đếm tập train/test

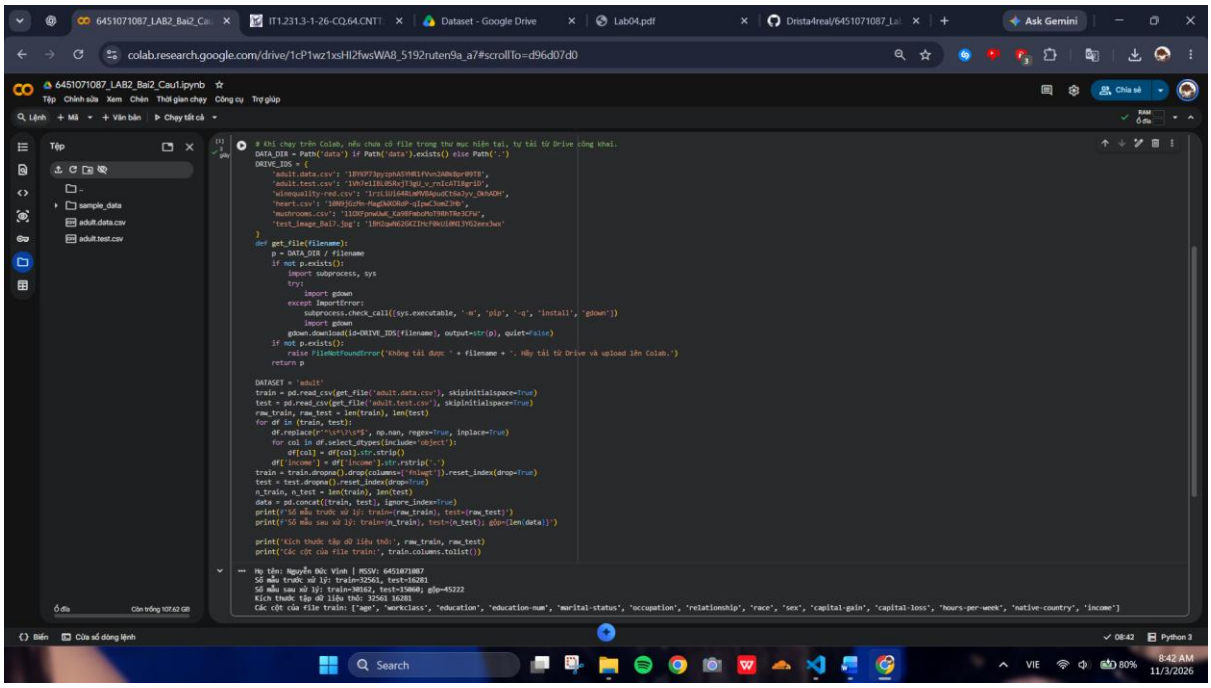
Notebook: 6451071087_LAB2_Bai2_Cau1.ipynb

Mã nguồn chính:

```
raw_train, raw_test = len(train), len(test)
for df in (train, test):
    df.replace(r'^\s*\?\s*$', np.nan, regex=True, inplace=True)
    for col in df.select_dtypes(include='object'):
        df[col] = df[col].str.strip()
    df['income'] = df['income'].str.rstrip('.')
train = train.dropna().drop(columns=['fnlwgt']).reset_index(drop=True)
test = test.dropna().reset_index(drop=True)
n_train, n_test = len(train), len(test)
data = pd.concat([train, test], ignore_index=True)
print(f'Số mẫu trước xử lý: train={raw_train}, test={raw_test}')
print(f'Số mẫu sau xử lý: train={n_train}, test={n_test}; gộp={len(data)}')


# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Kích thước tập dữ liệu thô: 32561 16281

Câu 2: Làm sạch, loại fnlwgt, nối train/test

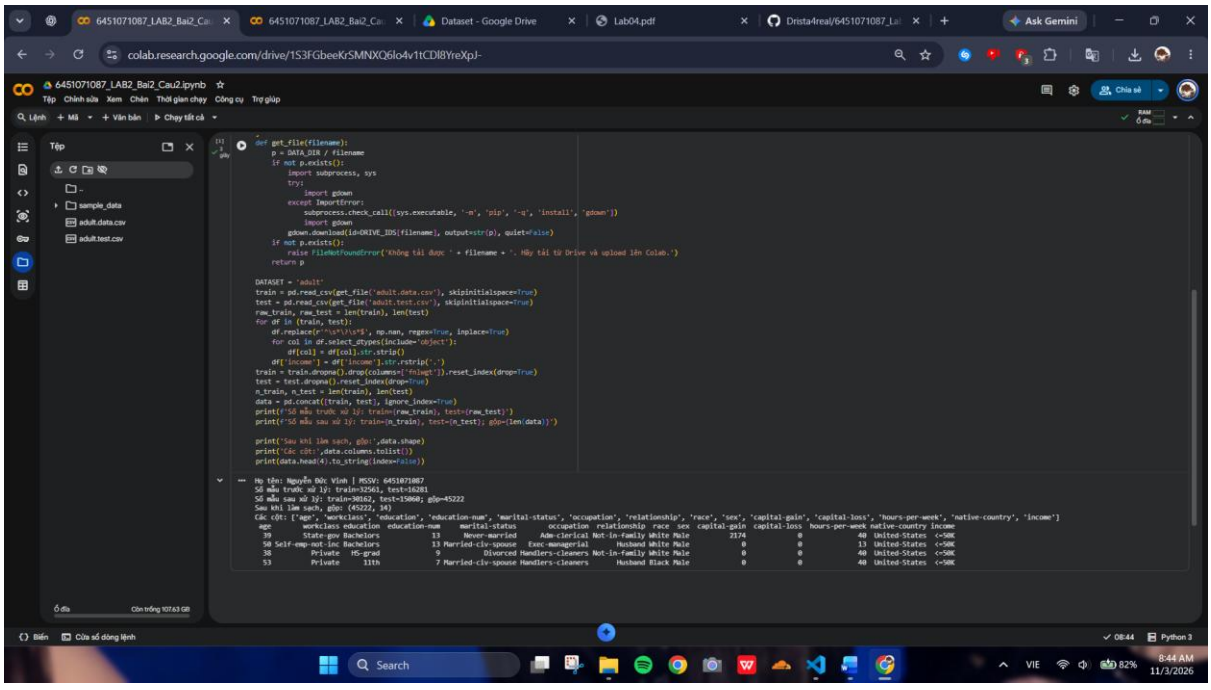
Notebook: 6451071087_LAB2_Bai2_Cau2.ipynb

Mã nguồn chính:

```
for df in (train, test):
    df.replace(r'^\s*\?\s*$', np.nan, regex=True, inplace=True)
    for col in df.select_dtypes(include='object'):
        df[col] = df[col].str.strip()
    df['income'] = df['income'].str.rstrip('.')
train = train.dropna().drop(columns=['fnlwgt']).reset_index(drop=True)
test = test.dropna().reset_index(drop=True)
n_train, n_test = len(train), len(test)
data = pd.concat([train, test], ignore_index=True)
print(f'Số mẫu trước xử lý: train={raw_train}, test={raw_test}')
print(f'Số mẫu sau xử lý: train={n_train}, test={n_test}; gộp={len(data)}')


print('Sau khi làm sạch, gộp:',data.shape)
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Các cột: ['age', 'workclass', 'education', 'education-num', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'capita

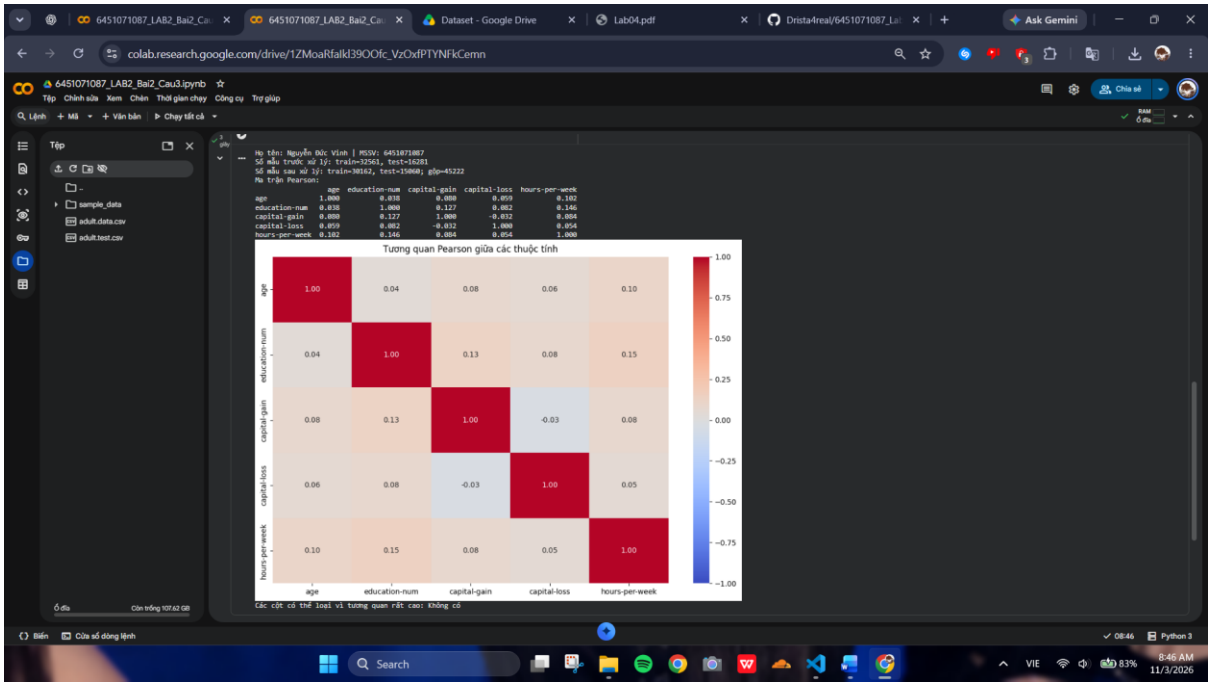
Câu 3: Hệ số tương quan Pearson

Notebook: 6451071087_LAB2_Bai2_Cau3.ipynb

Mã nguồn chính:

```
numeric = data.drop(columns=['income']).select_dtypes(include=np.number)
corr = numeric.corr(method='pearson')
print('Ma trận Pearson:')
print(corr.round(3).to_string())
plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Tương quan Pearson giữa các thuộc tính')
plt.tight_layout(); plt.show()
# Ngưỡng tham khảo |r| > 0.90, xét một nửa tam giác trên để tránh trùng lặp.
upper = corr.abs().where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_corr = [c for c in upper.columns if any(upper[c] > 0.90)]
print('Các cột có thể loại vì tương quan rất cao:', drop_corr if drop_corr else 'Không có')
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Các cột có thể loại vì tương quan rất cao: Không có

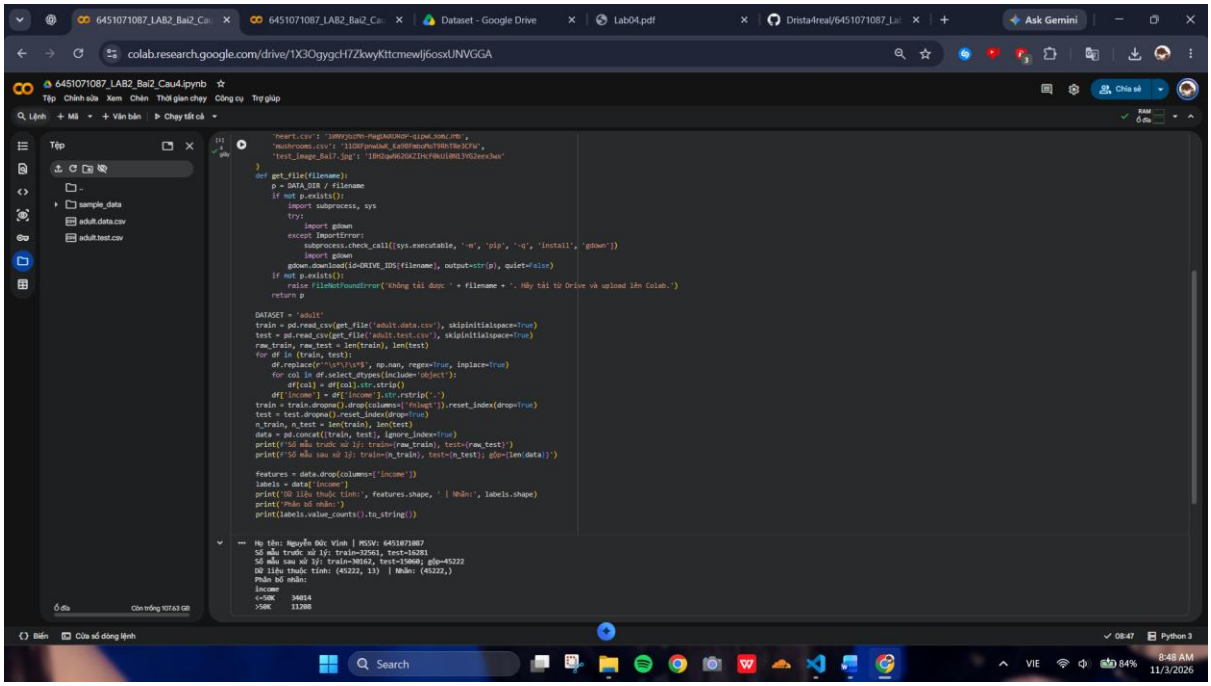
Câu 4: Tách thuộc tính và nhãn

Notebook: 6451071087_LAB2_Bai2_Cau4.ipynb

Mã nguồn chính:

```
features = data.drop(columns=['income'])
labels = data['income']
print('Dữ liệu thuộc tính:', features.shape, ' | Nhãn:', labels.shape)
print('Phân bố nhãn:')
print(labels.value_counts().to_string())
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Dữ liệu thuộc tính: (45222, 13) | Nhãn: (45222,)

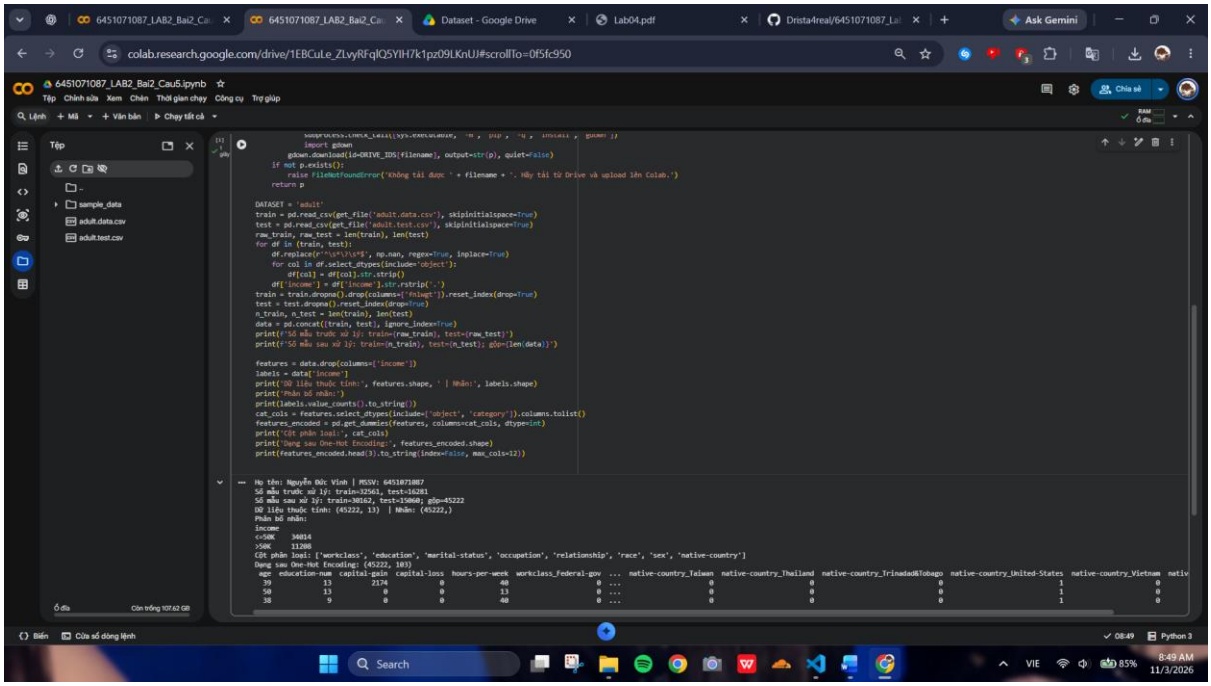
Câu 5: One-hot encoding

Notebook: 6451071087_LAB2_Bai2_Cau5.ipynb

Mã nguồn chính:

```
cat_cols = features.select_dtypes(include=['object', 'category']).columns.tolist()
features_encoded = pd.get_dummies(features, columns=cat_cols, dtype=int)
print('Cột phân loại:', cat_cols)
print('Dạng sau One-Hot Encoding:', features_encoded.shape)
print(features_encoded.head(3).to_string(index=False, max_cols=12))
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Dữ liệu thuộc tính: (45222, 13) | Nhãn: (45222,)
Cột phân loại: ['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']
Dạng sau One-Hot Encoding: (45222, 103)

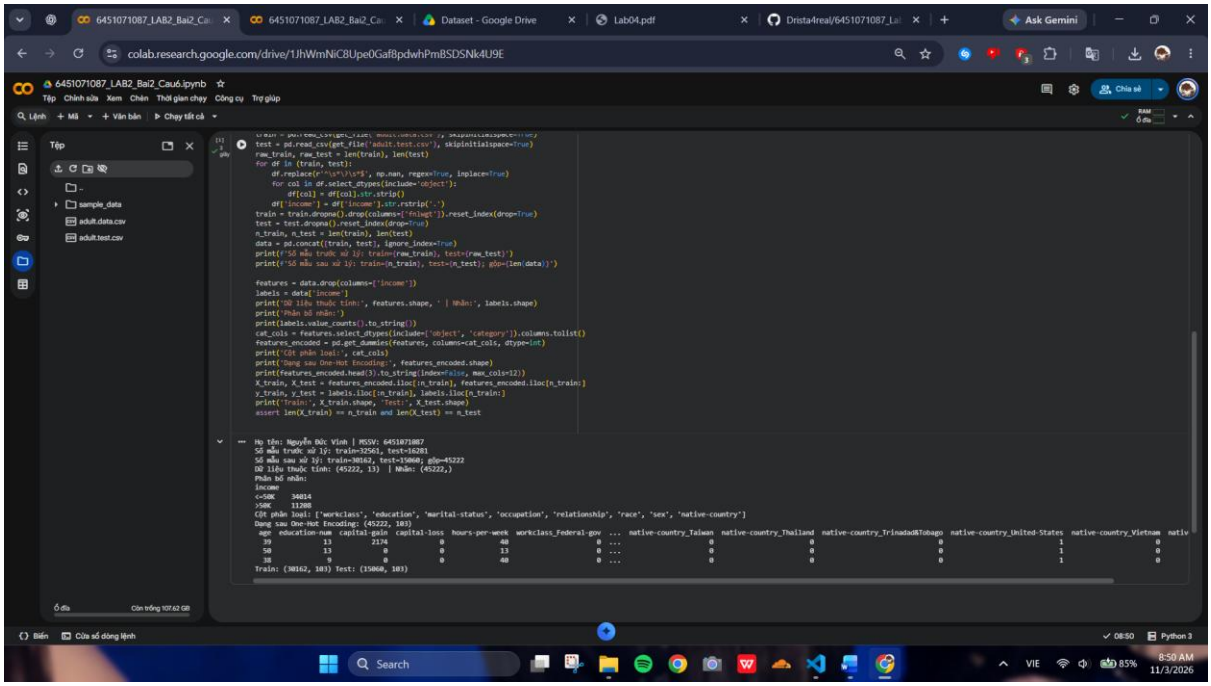
Câu 6: Tách train và test như ban đầu

Notebook: 6451071087_LAB2_Bai2_Cau6.ipynb

Mã nguồn chính:

```
X_train, X_test = features_encoded.iloc[:n_train], features_encoded.iloc[n_train:]
y_train, y_test = labels.iloc[:n_train], labels.iloc[n_train:]
print('Train:', X_train.shape, 'Test:', X_test.shape)
assert len(X_train) == n_train and len(X_test) == n_test
```

Kết quả thực tế:



Số mẫu trước xử lý: train=32561, test=16281
Số mẫu sau xử lý: train=30162, test=15060; gộp=45222
Dữ liệu thuộc tính: (45222, 13) | Nhãn: (45222,)
Cột phân loại: ['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']
Dạng sau One-Hot Encoding: (45222, 103)
Train: (30162, 103) Test: (15060, 103)

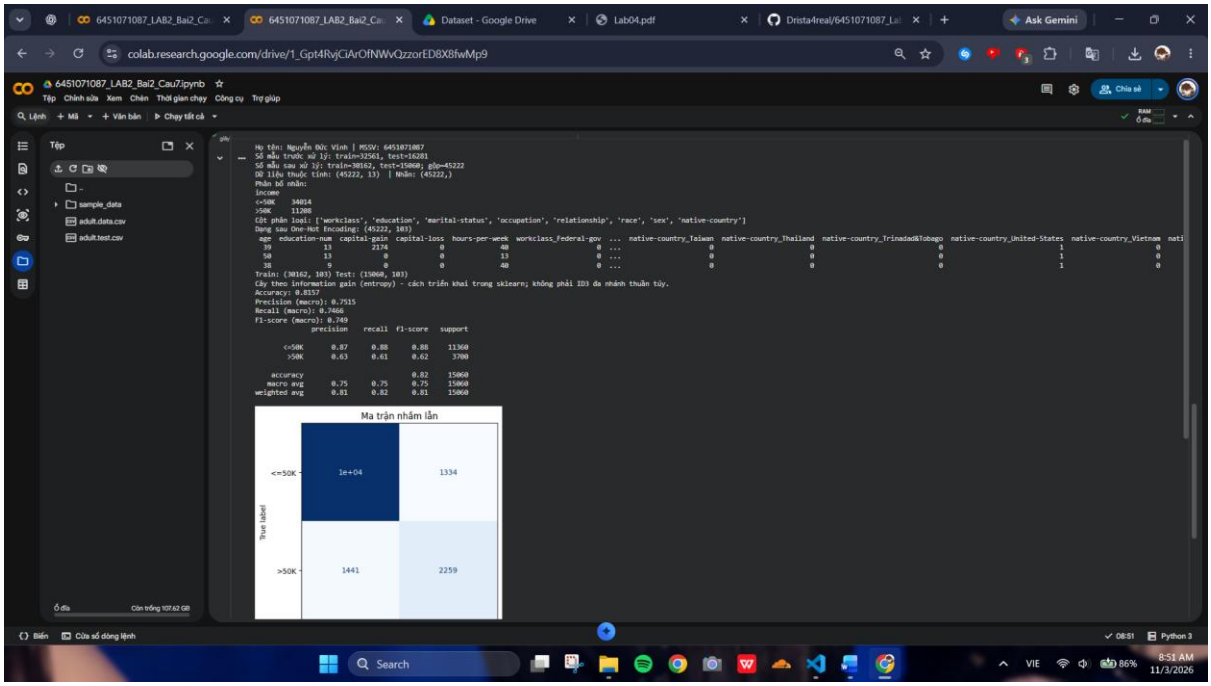
Câu 7: Cây quyết định entropy (ID3 theo hướng dẫn)

Notebook: 6451071087_LAB2_Bai2_Cau7.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây theo information gain (entropy) - cách triển khai trong sklearn; không phải ID3 đa nhánh thuần túy.')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False, xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (30162, 103) Test: (15060, 103)
Accuracy: 0.8157
Precision (macro): 0.7515
Recall (macro): 0.7466
F1-score (macro): 0.749

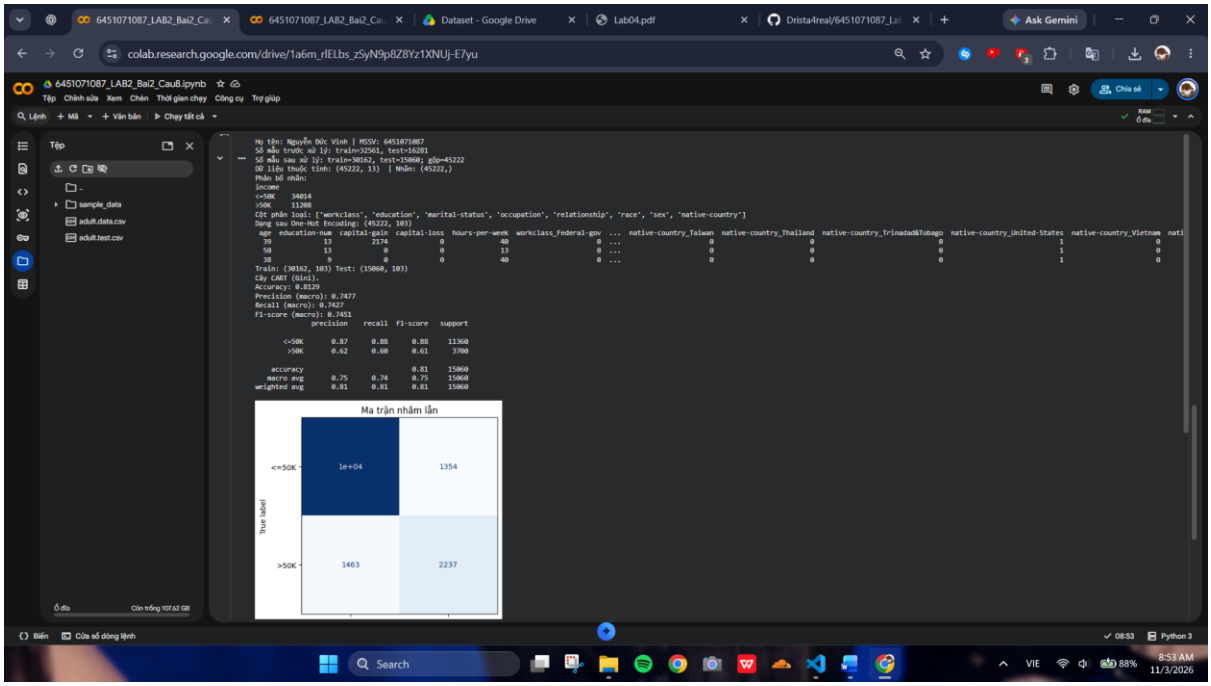
Câu 8: Cây CART (Gini)

Notebook: 6451071087_LAB2_Bai2_Cau8.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây CART (Gini).')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (30162, 103) Test: (15060, 103)
Accuracy: 0.8129
Precision (macro): 0.7477
Recall (macro): 0.7427
F1-score (macro): 0.7451

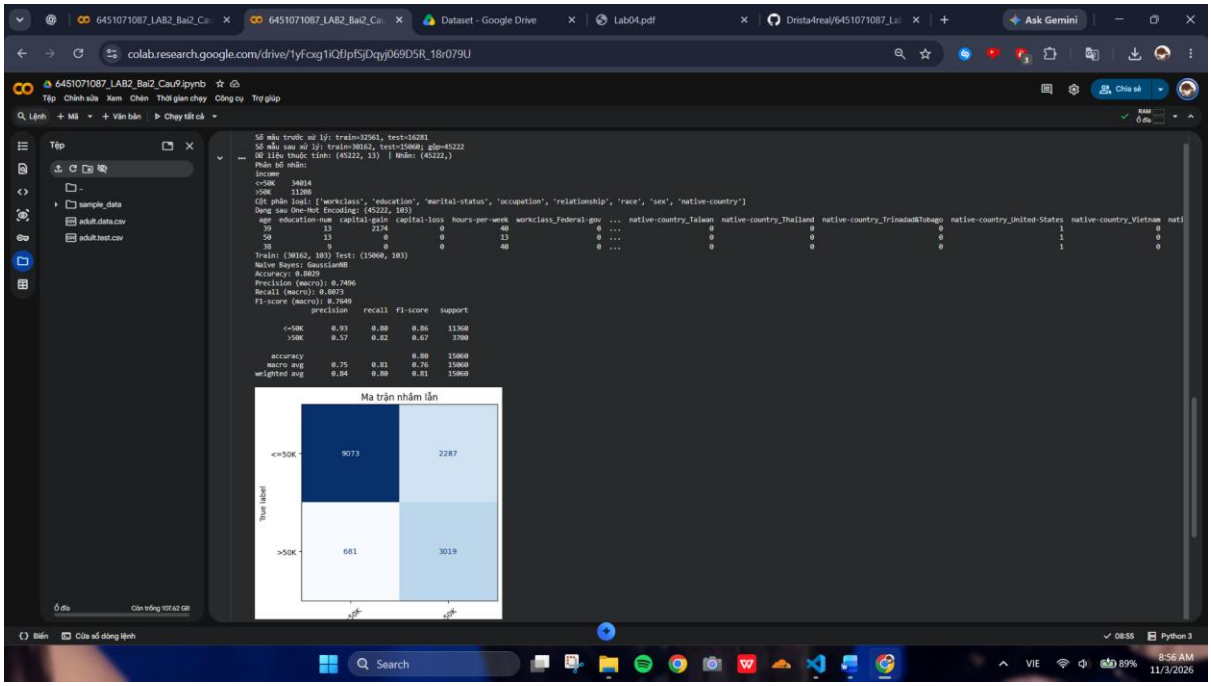
Câu 9: Naive Bayes

Notebook: 6451071087_LAB2_Bai2_Cau9.ipynb

Mã nguồn chính:

```
model = MultinomialNB(alpha=1.0) if DATASET == 'mush' else GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Naïve Bayes:', type(model).__name__)
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (30162, 103) Test: (15060, 103)
Naïve Bayes: GaussianNB
Accuracy: 0.8029
Precision (macro): 0.7496
Recall (macro): 0.8073
F1-score (macro): 0.7649

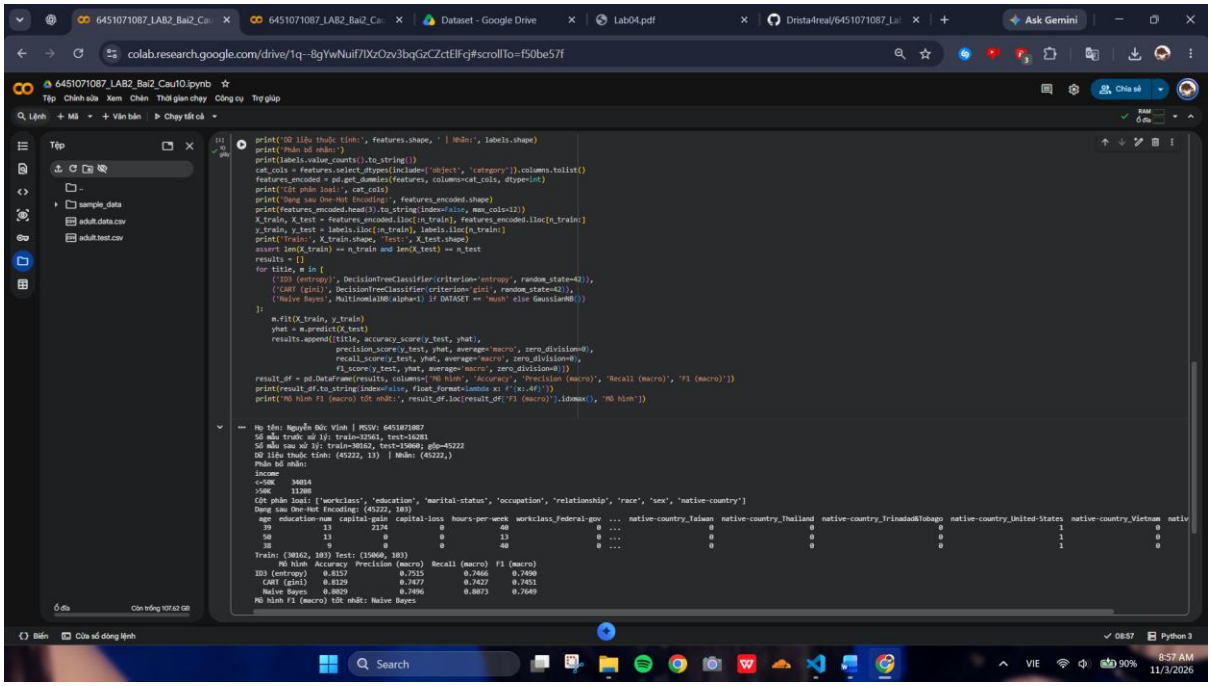
Câu 10: So sánh ba mô hình

Notebook: 6451071087_LAB2_Bai2_Cau10.ipynb

Mã nguồn chính:

```
results = []
for title, m in [
    ('ID3 (entropy)', DecisionTreeClassifier(criterion='entropy', random_state=42)),
    ('CART (gini)', DecisionTreeClassifier(criterion='gini', random_state=42)),
    ('Naive Bayes', MultinomialNB(alpha=1) if DATASET == 'mush' else GaussianNB())
]:
    m.fit(X_train, y_train)
    yhat = m.predict(X_test)
    results.append([title, accuracy_score(y_test, yhat),
                    precision_score(y_test, yhat, average='macro', zero_division=0),
                    recall_score(y_test, yhat, average='macro', zero_division=0),
                    f1_score(y_test, yhat, average='macro', zero_division=0)])
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Mô hình	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
ID3 (entropy)	0.8157	0.7515	0.7466	0.7490
CART (gini)	0.8129	0.7477	0.7427	0.7451
Naive Bayes	0.8029	0.7496	0.8073	0.7649

Mô hình F1 (macro) tốt nhất: Naive Bayes

Nhận xét: Ưu tiên so sánh F1-macro khi dữ liệu có nhiều lớp hoặc các lớp mất cân bằng. Accuracy và Precision/Recall được ghi cụ thể phía trên; kết quả đánh giá chỉ phản ánh cách chia tập dữ liệu đã chọn.

Bài 4 (Trang 26) – Red Wine Quality

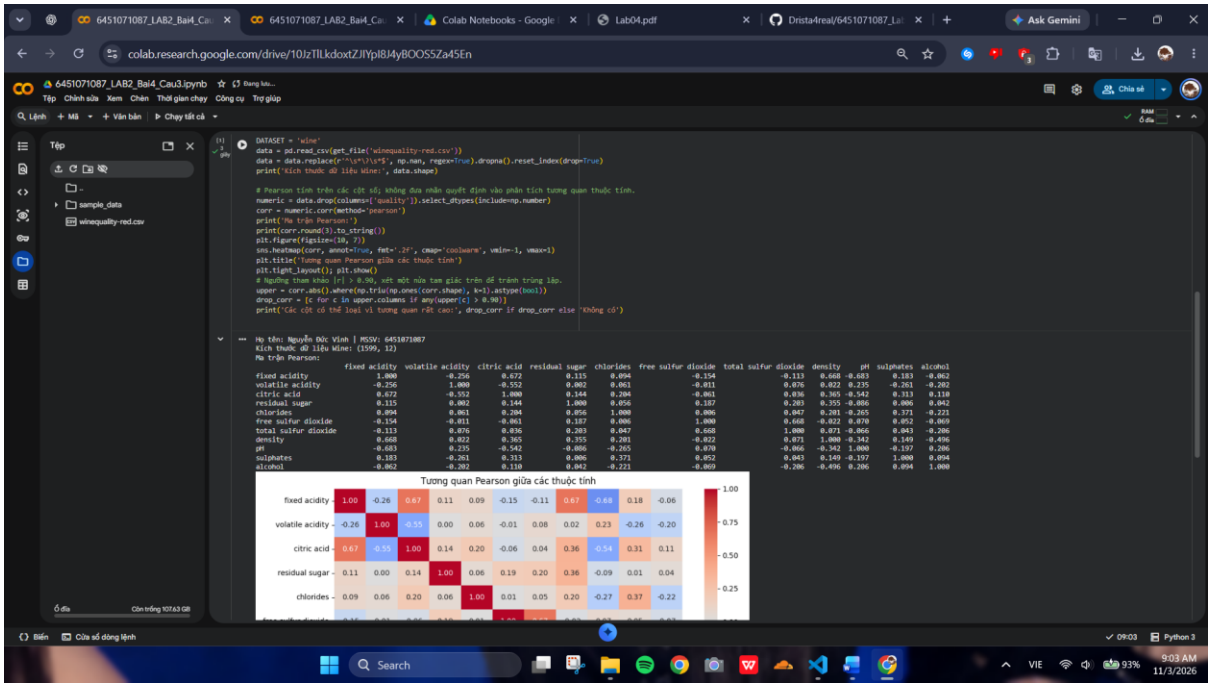
Câu 3: Pearson trên Red Wine Quality

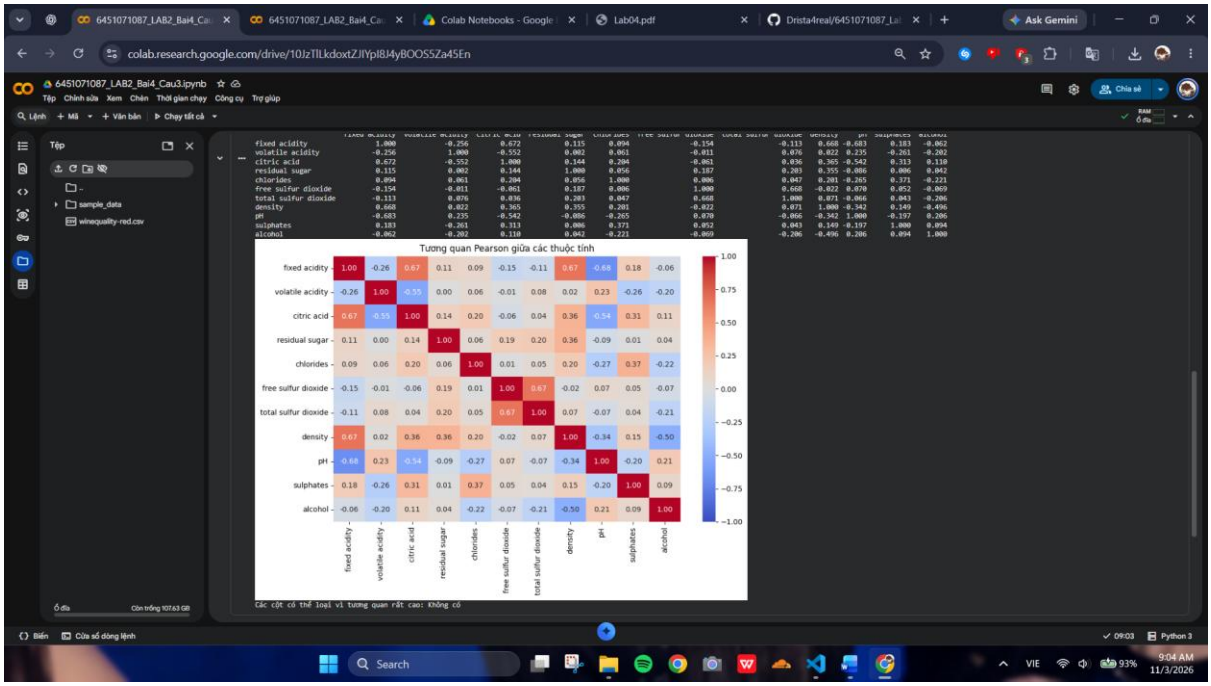
Notebook: 6451071087_LAB2_Bai4_Cau3.ipynb

Mã nguồn chính:

```
numeric = data.drop(columns=['quality']).select_dtypes(include=np.number)
corr = numeric.corr(method='pearson')
print('Ma trận Pearson:')
print(corr.round(3).to_string())
plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Tương quan Pearson giữa các thuộc tính')
plt.tight_layout(); plt.show()
# Ngưỡng tham khảo |r| > 0.90, xét một nửa tam giác trên để tránh trùng lặp.
upper = corr.abs().where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_corr = [c for c in upper.columns if any(upper[c] > 0.90)]
print('Các cột có thể loại vì tương quan rất cao:', drop_corr if drop_corr else 'Không có')
```

Kết quả thực tế:





Kích thước dữ liệu Wine: (1599, 12)
Các cột có thể loại vì tương quan rất cao: Không có

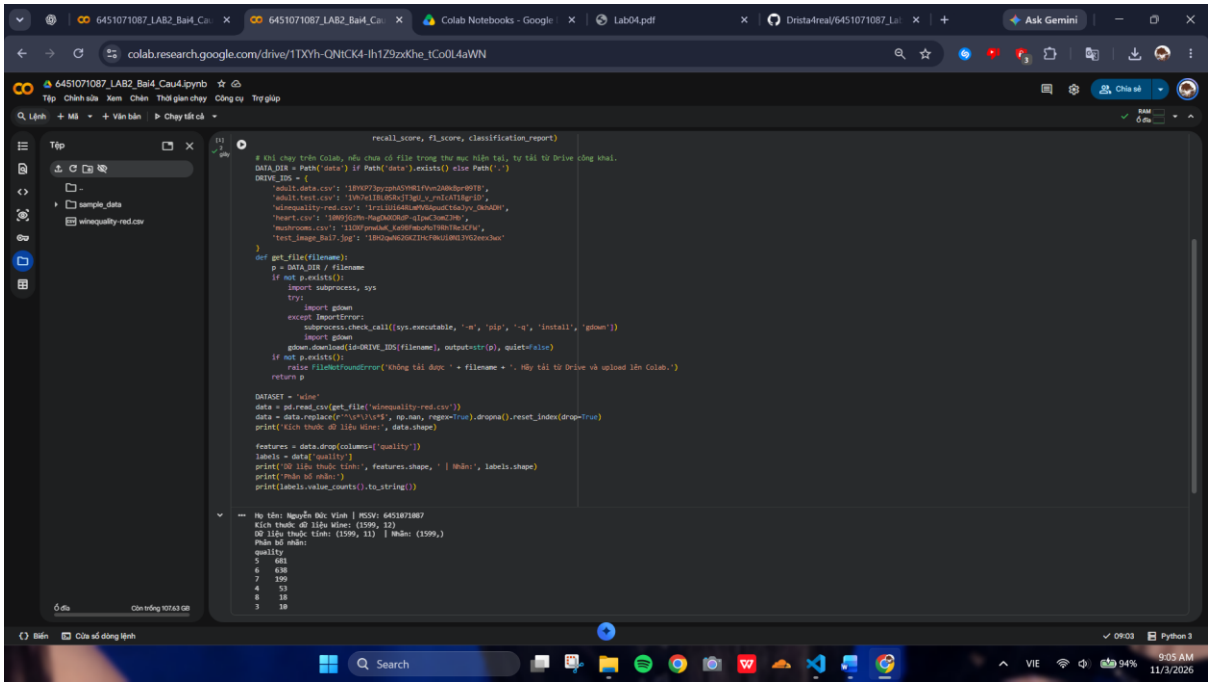
Câu 4: Tách thuộc tính và quality

Notebook: 6451071087_LAB2_Bai4_Cau4.ipynb

Mã nguồn chính:

```
features = data.drop(columns=['quality'])
labels = data['quality']
print('Dữ liệu thuộc tính:', features.shape, ' | Nhãn:', labels.shape)
print('Phân bố nhãn:')
print(labels.value_counts().to_string())
```

Kết quả thực tế:



Kích thước dữ liệu Wine: (1599, 12)
Dữ liệu thuộc tính: (1599, 11) | Nhãn: (1599,)

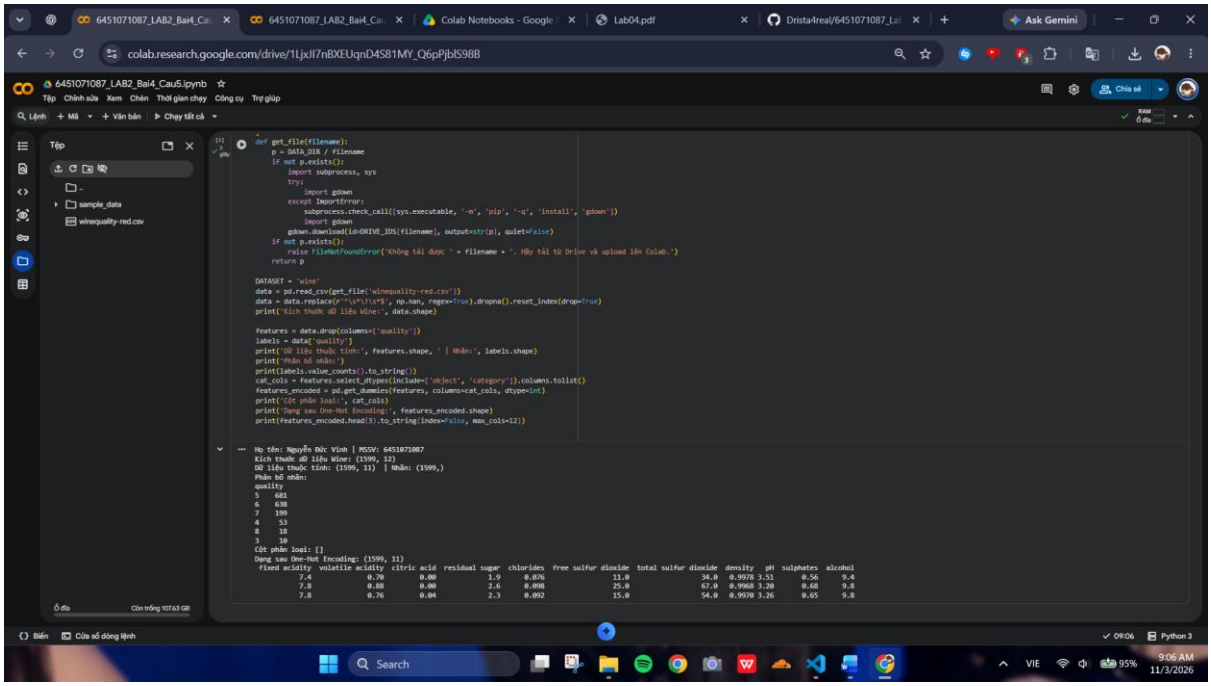
Câu 5: Mã hóa cột phân loại nếu có

Notebook: 6451071087_LAB2_Bai4_Cau5.ipynb

Mã nguồn chính:

```
cat_cols = features.select_dtypes(include=['object', 'category']).columns.tolist()
features_encoded = pd.get_dummies(features, columns=cat_cols, dtype=int)
print('Cột phân loại:', cat_cols)
print('Dạng sau One-Hot Encoding:', features_encoded.shape)
print(features_encoded.head(3).to_string(index=False, max_cols=12))
```

Kết quả thực tế:



Kích thước dữ liệu Wine: (1599, 12)
Dữ liệu thuộc tính: (1599, 11) | Nhãn: (1599,)
Cột phân loại: []
Dạng sau One-Hot Encoding: (1599, 11)

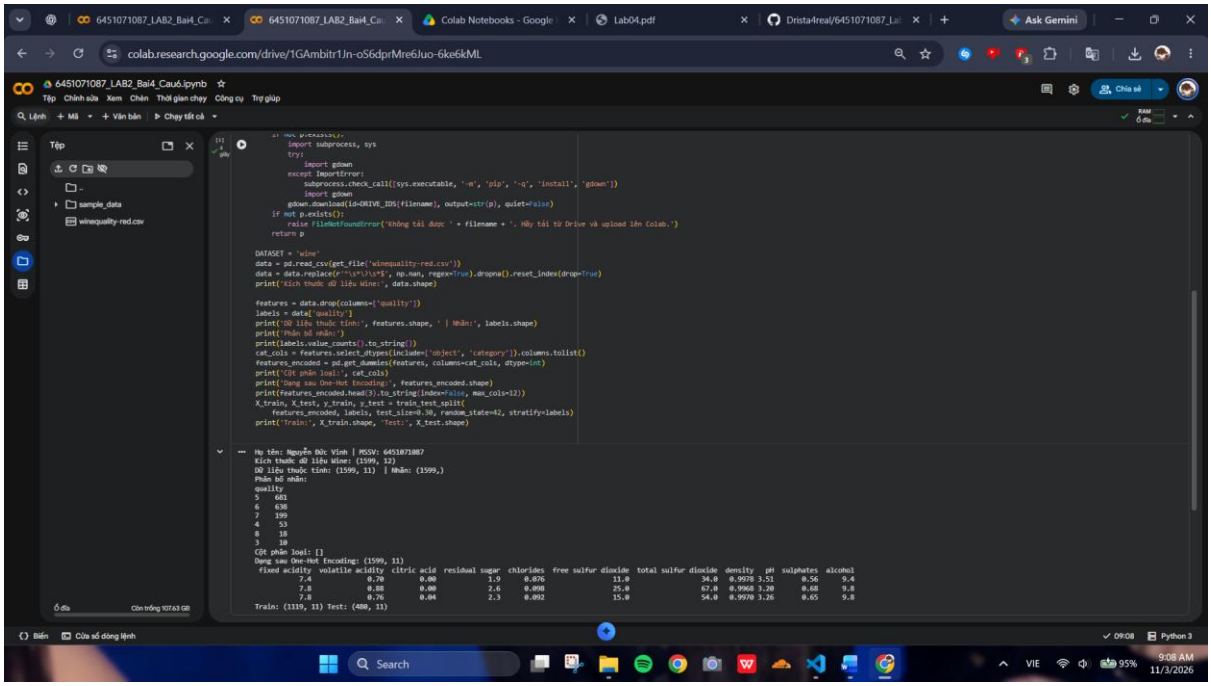
Câu 6: Chia train/test 70/30

Notebook: 6451071087_LAB2_Bai4_Cau6.ipynb

Mã nguồn chính:

```
X_train, X_test, y_train, y_test = train_test_split(
    features_encoded, labels, test_size=0.30, random_state=42, stratify=labels)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

Kết quả thực tế:



Kích thước dữ liệu Wine: (1599, 12)
Dữ liệu thuộc tính: (1599, 11) | Nhãn: (1599,)
Cột phân loại: []
Dạng sau One-Hot Encoding: (1599, 11)
Train: (1119, 11) Test: (480, 11)

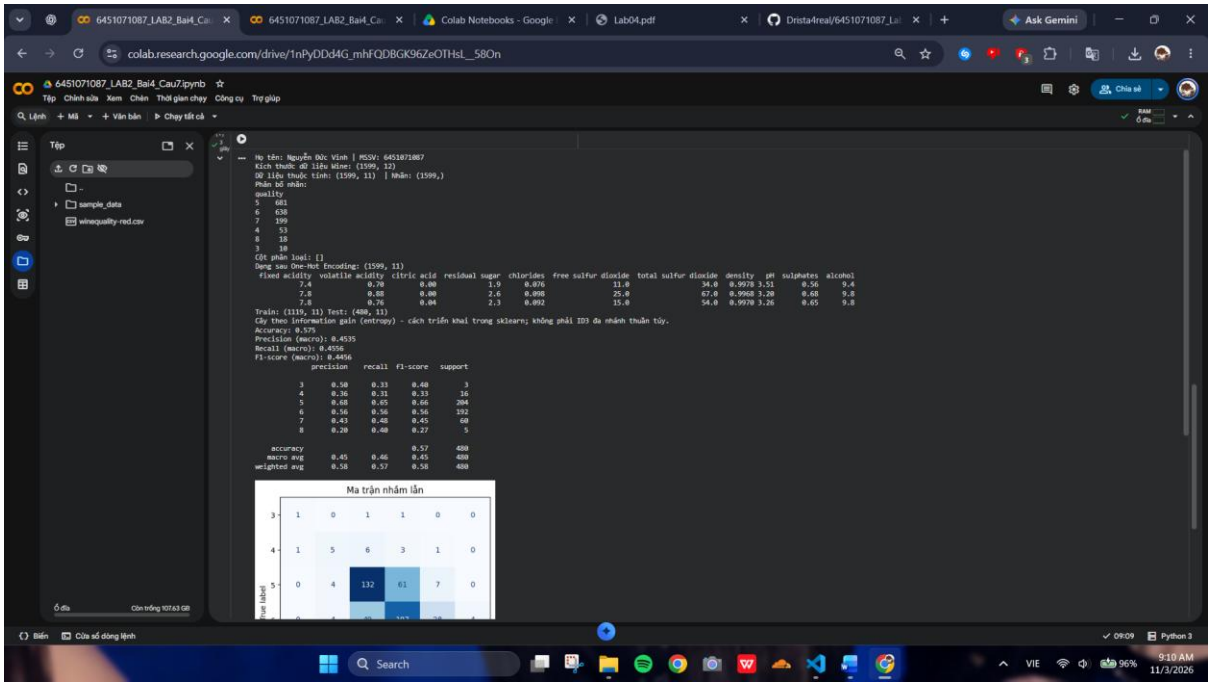
Câu 7: Cây entropy

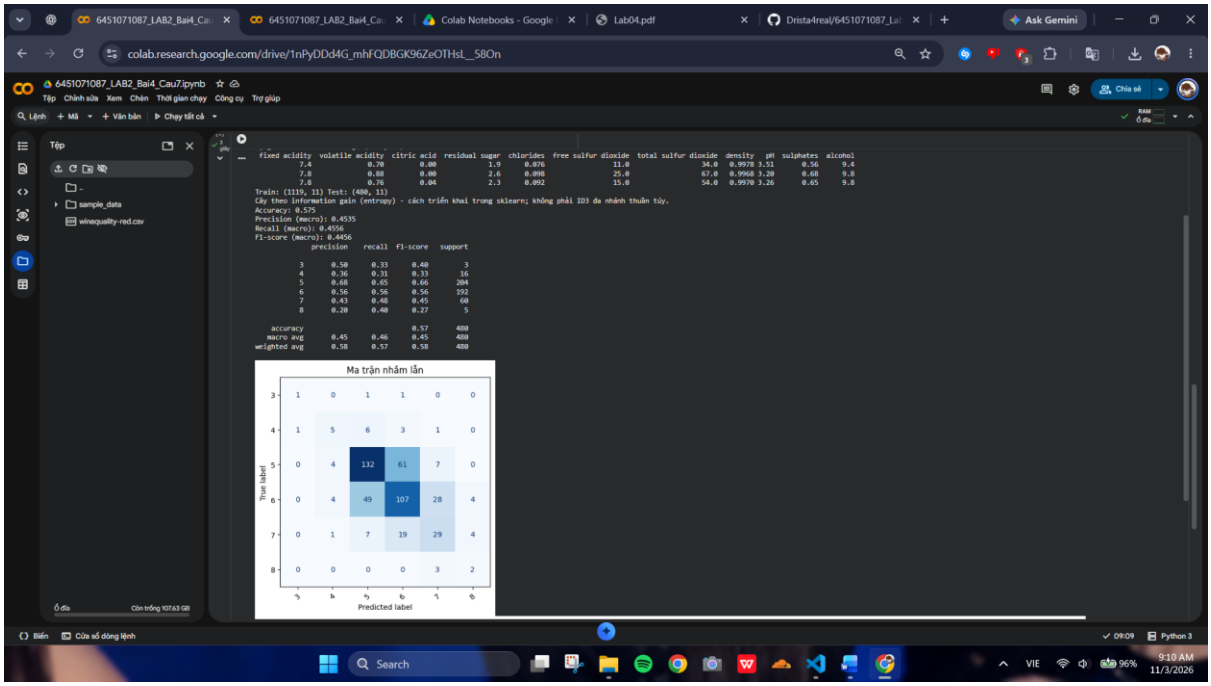
Notebook: 6451071087_LAB2_Bai4_Cau7.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây theo information gain (entropy) - cách triển khai trong sklearn; không phải ID3 đa nhánh thuần
túy.')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:





Train: (1119, 11) Test: (480, 11)
Accuracy: 0.575
Precision (macro): 0.4535
Recall (macro): 0.4556
F1-score (macro): 0.4456

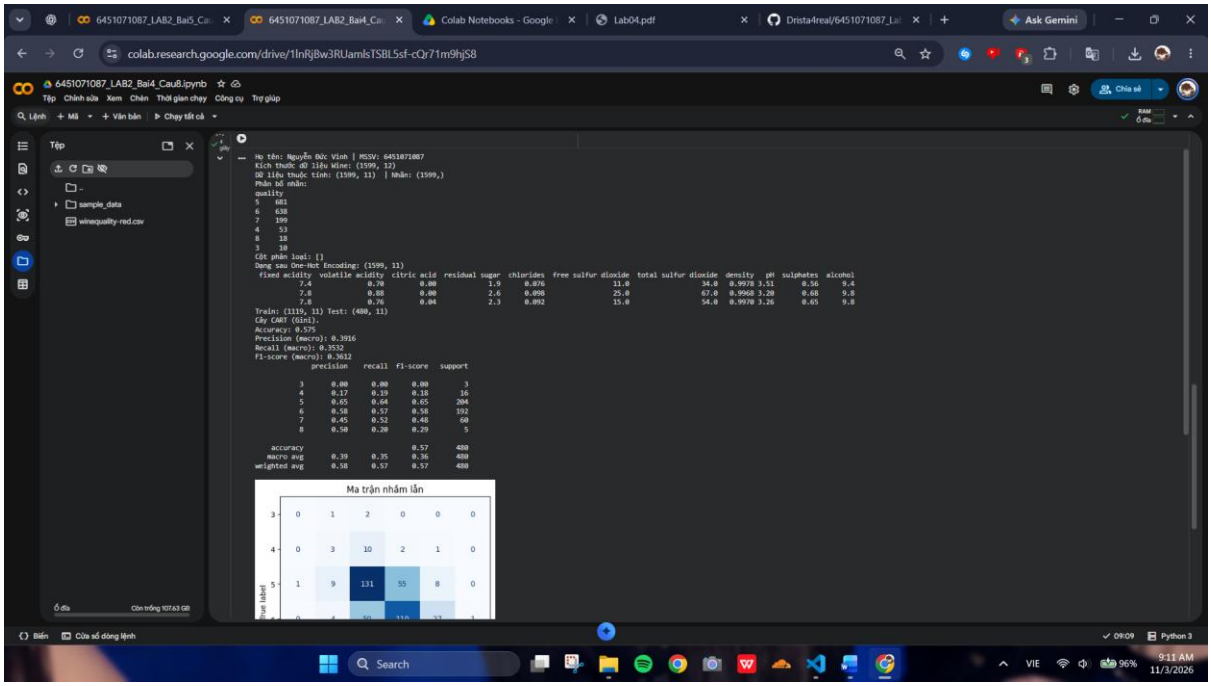
Câu 8: Cây Gini

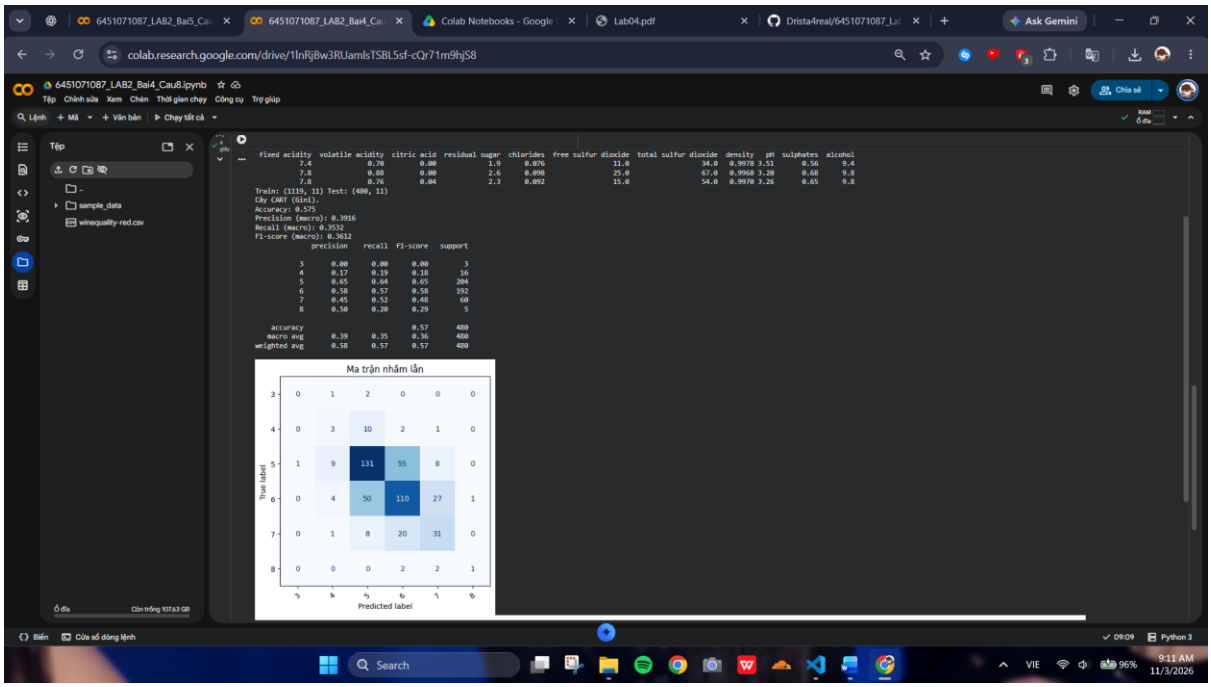
Notebook: 6451071087_LAB2_Bai4_Cau8.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây CART (Gini).')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:





Train: (1119, 11) Test: (480, 11)
Accuracy: 0.575
Precision (macro): 0.3916
Recall (macro): 0.3532
F1-score (macro): 0.3612

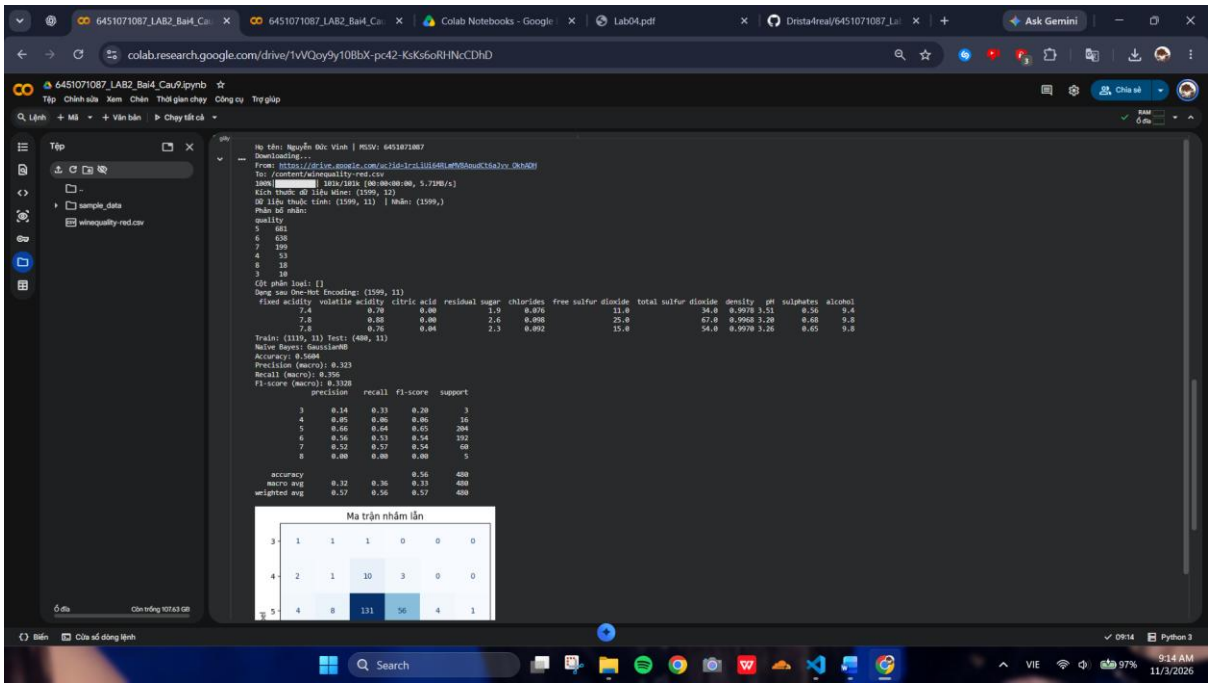
Câu 9: Naive Bayes

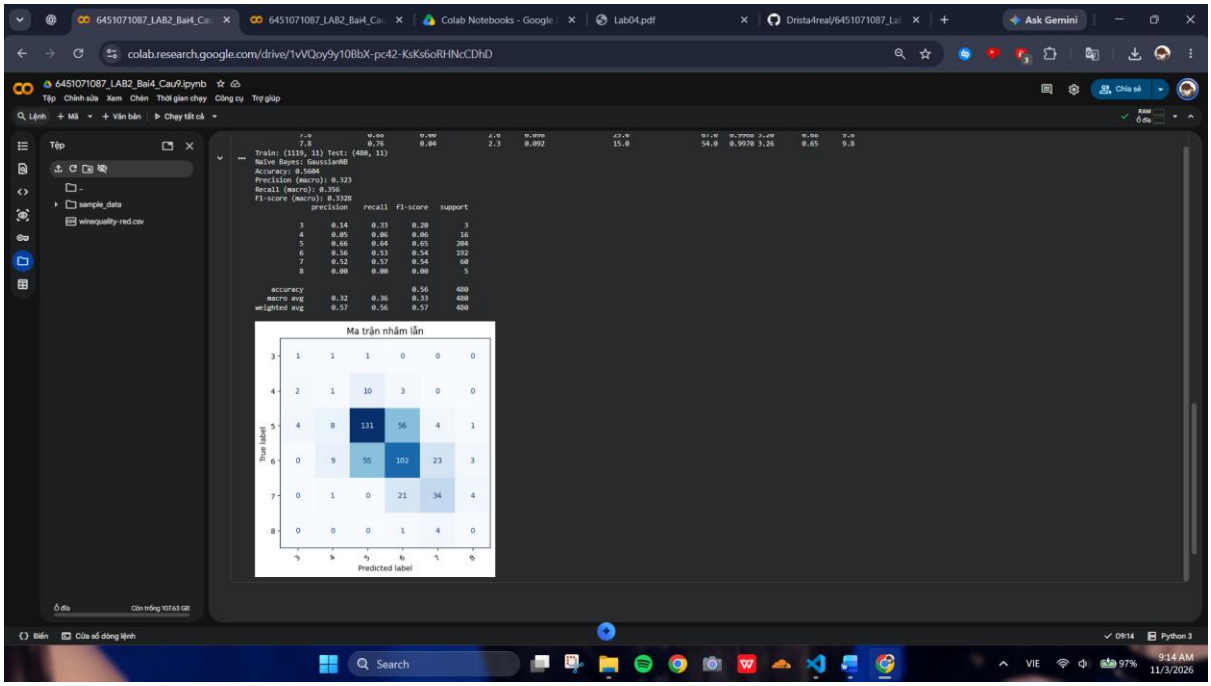
Notebook: 6451071087_LAB2_Bai4_Cau9.ipynb

Mã nguồn chính:

```
model = MultinomialNB(alpha=1.0) if DATASET == 'mush' else GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Naïve Bayes:', type(model).__name__)
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:





Train: (1119, 11) Test: (480, 11)
Naïve Bayes: GaussianNB
Accuracy: 0.5604
Precision (macro): 0.323
Recall (macro): 0.356
F1-score (macro): 0.3328

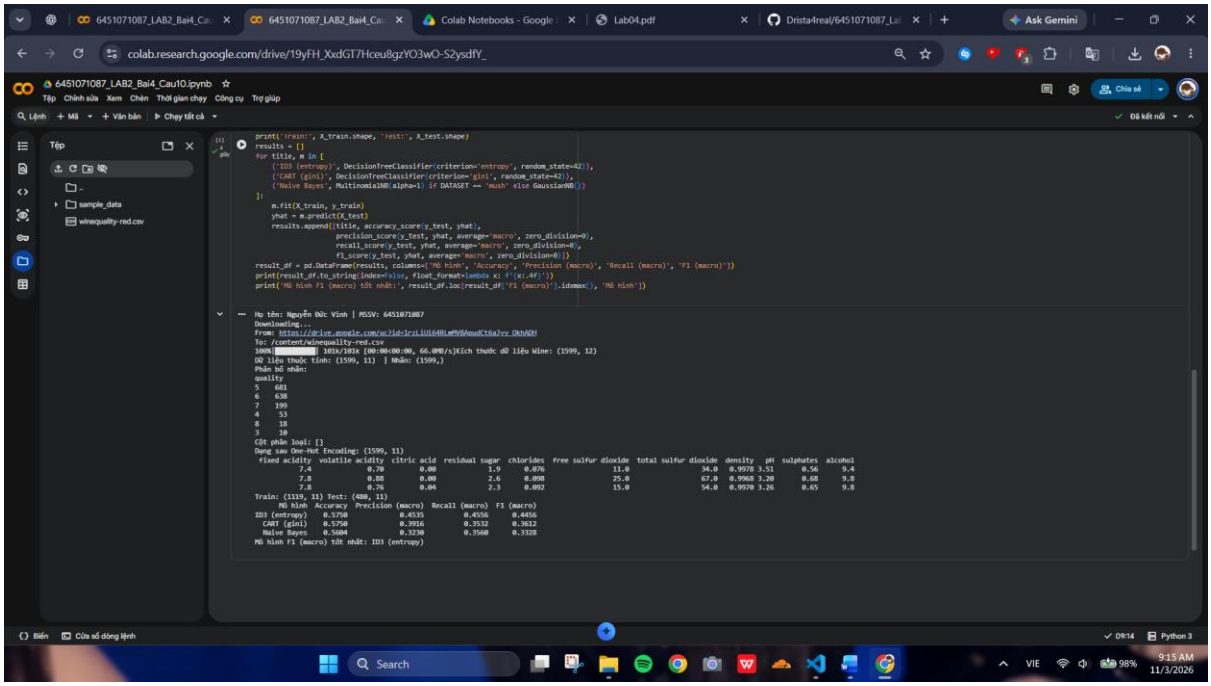
Câu 10: So sánh ID3, CART, NB

Notebook: 6451071087_LAB2_Bai4_Cau10.ipynb

Mã nguồn chính:

```
results = []
for title, m in [
    ('ID3 (entropy)', DecisionTreeClassifier(criterion='entropy', random_state=42)),
    ('CART (gini)', DecisionTreeClassifier(criterion='gini', random_state=42)),
    ('Naive Bayes', MultinomialNB(alpha=1) if DATASET == 'mush' else GaussianNB())
]:
    m.fit(X_train, y_train)
    yhat = m.predict(X_test)
    results.append([title, accuracy_score(y_test, yhat),
                    precision_score(y_test, yhat, average='macro', zero_division=0),
                    recall_score(y_test, yhat, average='macro', zero_division=0),
                    f1_score(y_test, yhat, average='macro', zero_division=0)])
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (1119, 11) Test: (480, 11)

Mô hình	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
ID3 (entropy)	0.5750	0.4535	0.4556	0.4456
CART (gini)	0.5750	0.3916	0.3532	0.3612
Naive Bayes	0.5604	0.3230	0.3560	0.3328

Mô hình F1 (macro) tốt nhất: ID3 (entropy)

Nhận xét: Ưu tiên so sánh F1-macro khi dữ liệu có nhiều lớp hoặc các lớp mất cân bằng. Accuracy và Precision/Recall được ghi cụ thể phía trên; kết quả đánh giá chỉ phản ánh cách chia tập dữ liệu đã chọn.

Bài 5 (Trang 26, 27) – Heart Disease

Câu a) Nạp, tiền xử lý, đổi tên 14 cột

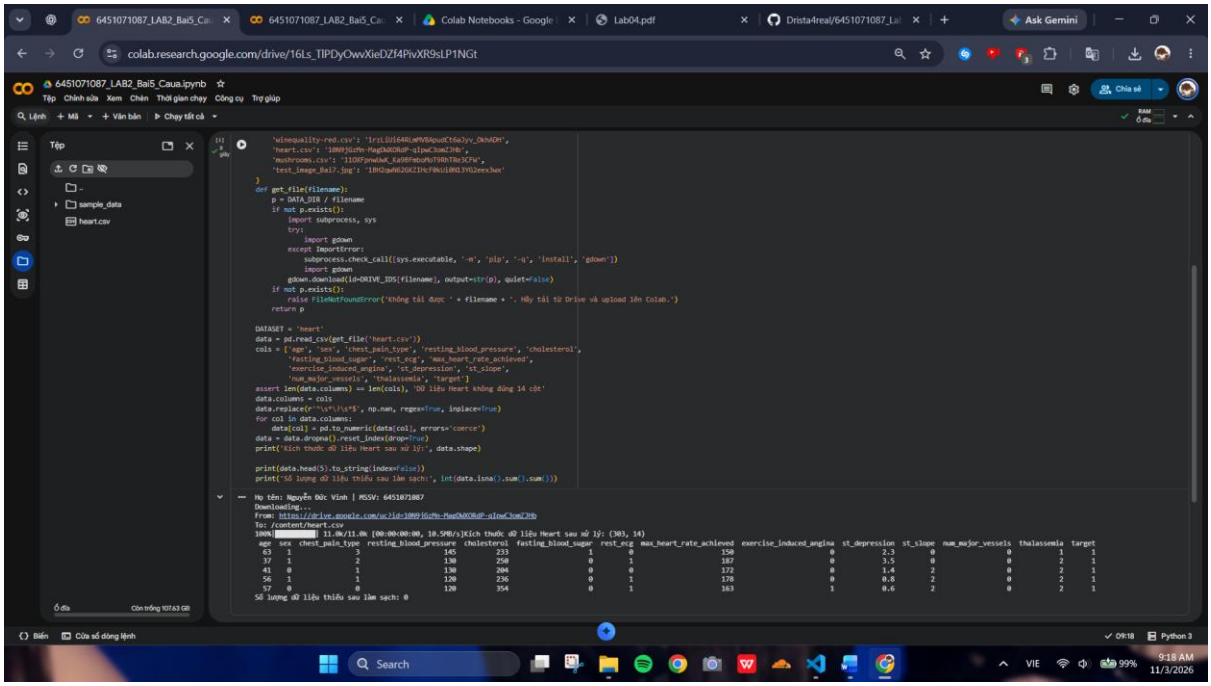
Notebook: 6451071087_LAB2_Bai5_Caua.ipynb

Mã nguồn chính:

```
cols = ['age', 'sex', 'chest_pain_type', 'resting_blood_pressure', 'cholesterol',
        'fasting_blood_sugar', 'rest_ecg', 'max_heart_rate_achieved',
        'exercise_induced_angina', 'st_depression', 'st_slope',
        'num_major_vessels', 'thalassemia', 'target']
assert len(data.columns) == len(cols), 'Dữ liệu Heart không đúng 14 cột'
data.columns = cols
data.replace(r'^\s*\?\s*$', np.nan, regex=True, inplace=True)
for col in data.columns:
    data[col] = pd.to_numeric(data[col], errors='coerce')
data = data.dropna().reset_index(drop=True)
print('Kích thước dữ liệu Heart sau xử lý:', data.shape)


print(data.head(5).to_string(index=False))
print('Số lượng dữ liệu thiếu sau làm sạch:', int(data.isna().sum().sum()))
```

Kết quả thực tế:



Kích thước dữ liệu Heart sau xử lý: (303, 14)
Số lượng dữ liệu thiếu sau làm sạch: 0

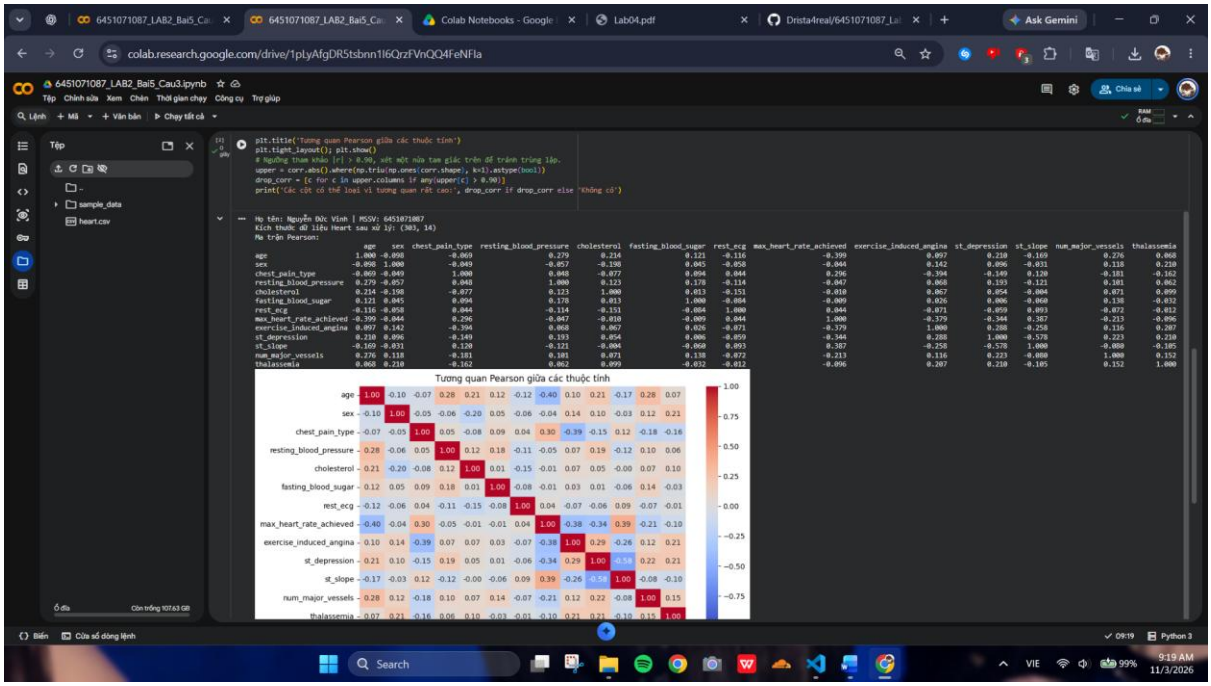
Câu 3: Pearson trên Heart Disease

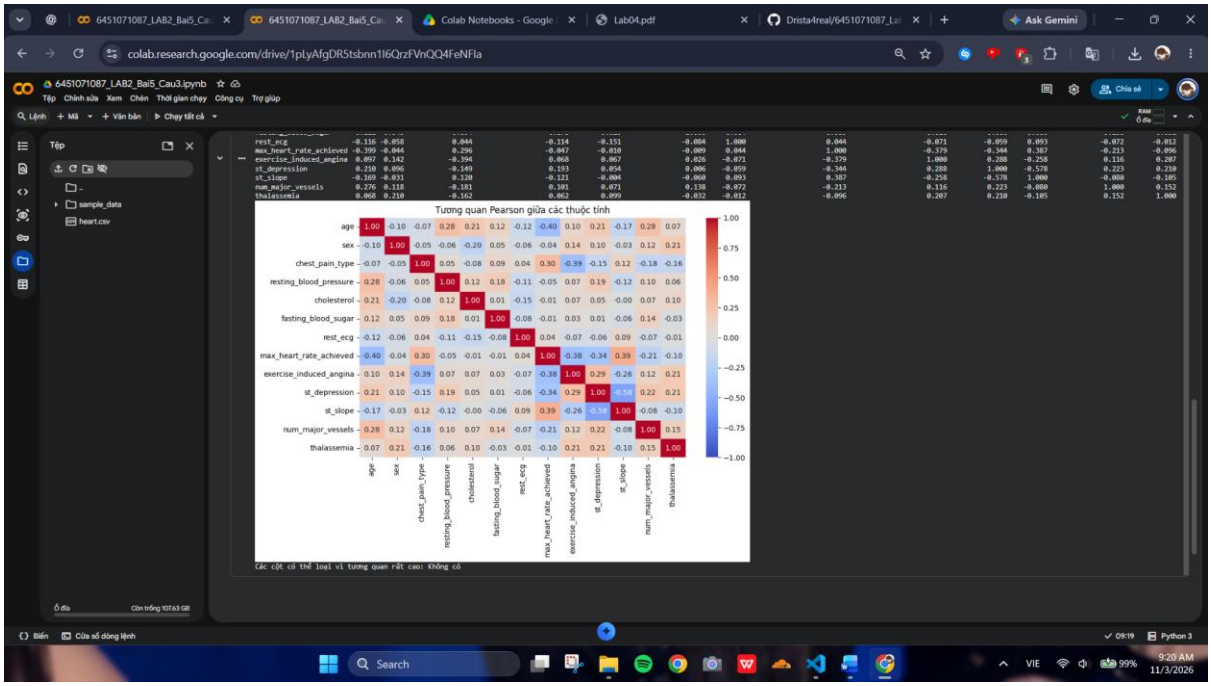
Notebook: 6451071087_LAB2_Bai5_Cau3.ipynb

Mã nguồn chính:

```
numeric = data.drop(columns=['target']).select_dtypes(include=np.number)
corr = numeric.corr(method='pearson')
print('Ma trận Pearson:')
print(corr.round(3).to_string())
plt.figure(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Tương quan Pearson giữa các thuộc tính')
plt.tight_layout(); plt.show()
# Ngưỡng tham khảo |r| > 0.90, xét một nửa tam giác trên để tránh trùng lặp.
upper = corr.abs().where(np.triu(np.ones(corr.shape), k=1).astype(bool))
drop_corr = [c for c in upper.columns if any(upper[c] > 0.90)]
print('Các cột có thể loại vì tương quan rất cao:', drop_corr if drop_corr else 'Không có')
```

Kết quả thực tế:





Kích thước dữ liệu Heart sau xử lý: (303, 14)
Các cột có thể loại vì tương quan rất cao: Không có

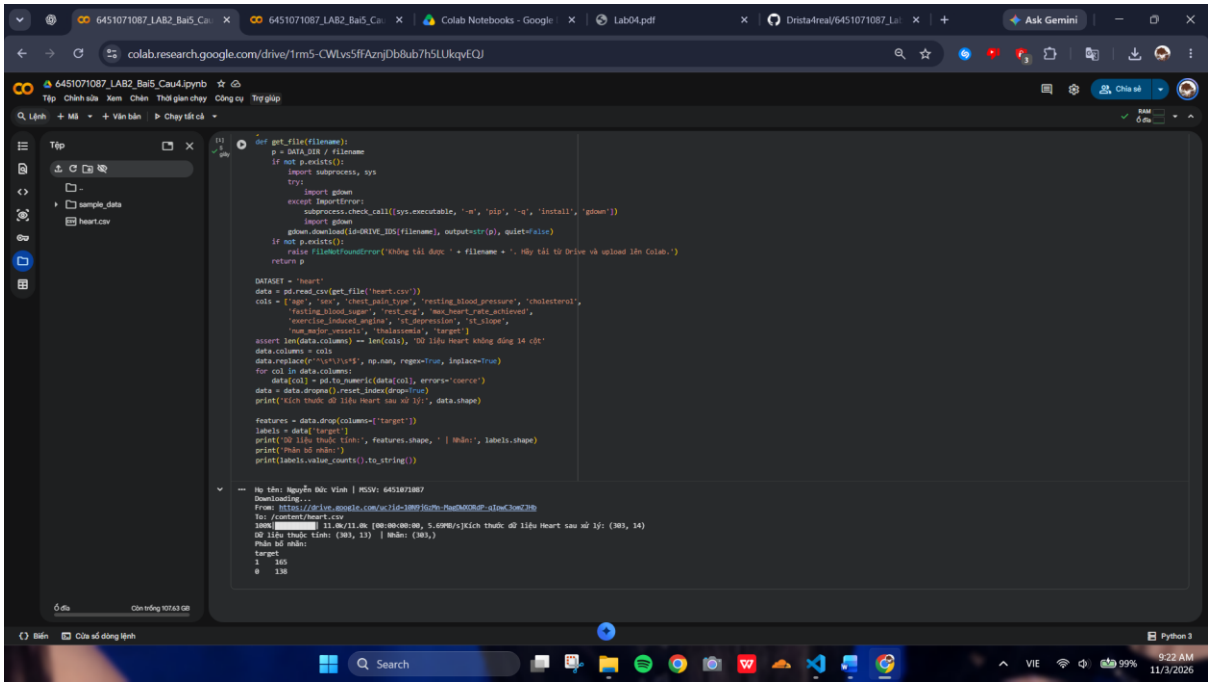
Câu 4: Tách thuộc tính và target

Notebook: 6451071087_LAB2_Bai5_Cau4.ipynb

Mã nguồn chính:

```
features = data.drop(columns=['target'])
labels = data['target']
print('Dữ liệu thuộc tính:', features.shape, ' | Nhãn:', labels.shape)
print('Phân bố nhãn:')
print(labels.value_counts().to_string())
```

Kết quả thực tế:



Kích thước dữ liệu Heart sau xử lý: (303, 14)
Dữ liệu thuộc tính: (303, 13) | Nhãn: (303,)

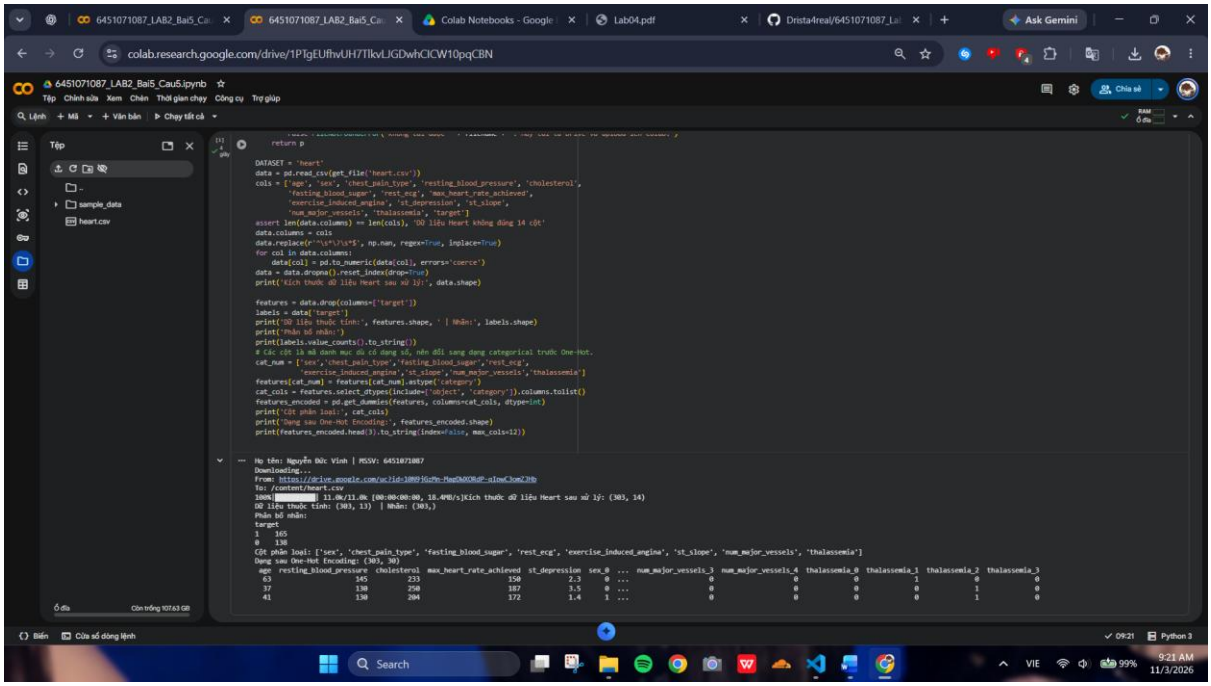
Câu 5: One-hot cho các biến danh mục

Notebook: 6451071087_LAB2_Bai5_Cau5.ipynb

Mã nguồn chính:

```
cat_num = ['sex','chest_pain_type','fasting_blood_sugar','rest_ecg',
           'exercise_induced_angina','st_slope','num_major_vessels','thalassemia']
features[cat_num] = features[cat_num].astype('category')
cat_cols = features.select_dtypes(include=['object', 'category']).columns.tolist()
features_encoded = pd.get_dummies(features, columns=cat_cols, dtype=int)
print('Cột phân loại:', cat_cols)
print('Dạng sau One-Hot Encoding:', features_encoded.shape)
print(features_encoded.head(3).to_string(index=False, max_cols=12))
```

Kết quả thực tế:



Kích thước dữ liệu Heart sau xử lý: (303, 14)
Dữ liệu thuộc tính: (303, 13) | Nhãn: (303,)
Cột phân loại: ['sex', 'chest_pain_type', 'fasting_blood_sugar', 'rest_ecg', 'exercise_induced_angina', 'st_slope', 'num_major_ves
Dạng sau One-Hot Encoding: (303, 30)

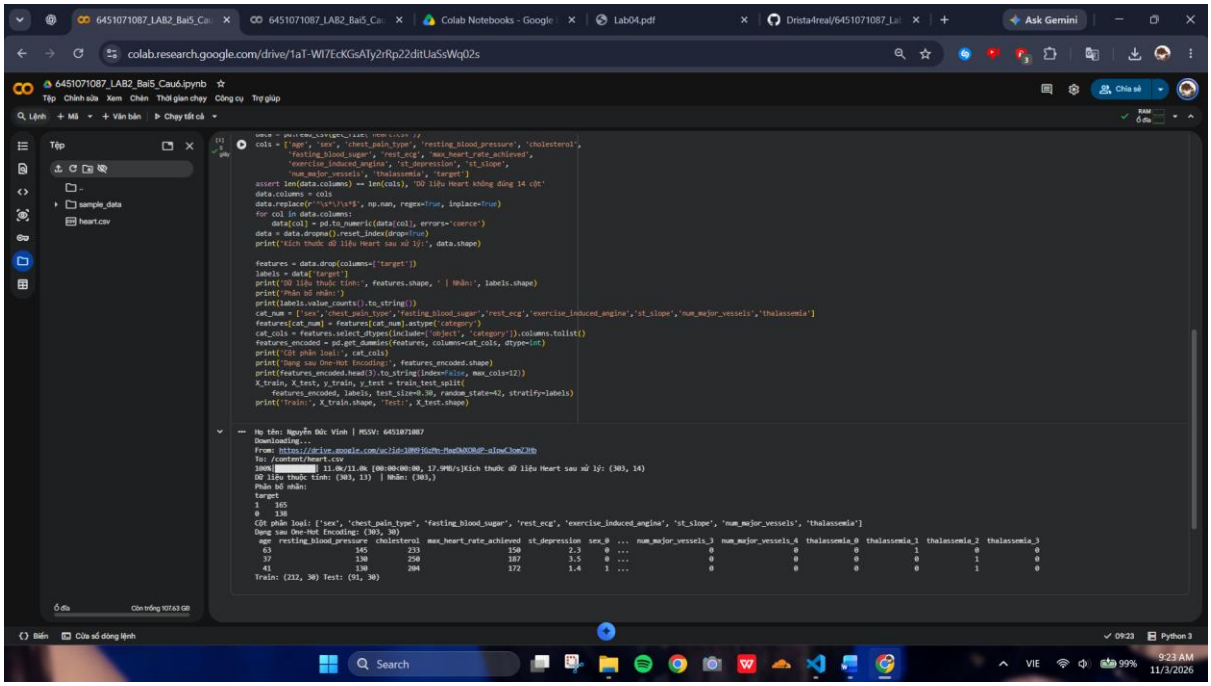
Câu 6: Chia train/test 70/30

Notebook: 6451071087_LAB2_Bai5_Cau6.ipynb

Mã nguồn chính:

```
X_train, X_test, y_train, y_test = train_test_split(
    features_encoded, labels, test_size=0.30, random_state=42, stratify=labels)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

Kết quả thực tế:



Kích thước dữ liệu Heart sau xử lý: (303, 14)
Dữ liệu thuộc tính: (303, 13) | Nhãn: (303,)
Cột phân loại: ['sex', 'chest_pain_type', 'fasting_blood_sugar', 'rest_ecg', 'exercise_induced_angina', 'st_slope', 'num_major_ves
Dạng sau One-Hot Encoding: (303, 30)
Train: (212, 30) Test: (91, 30)

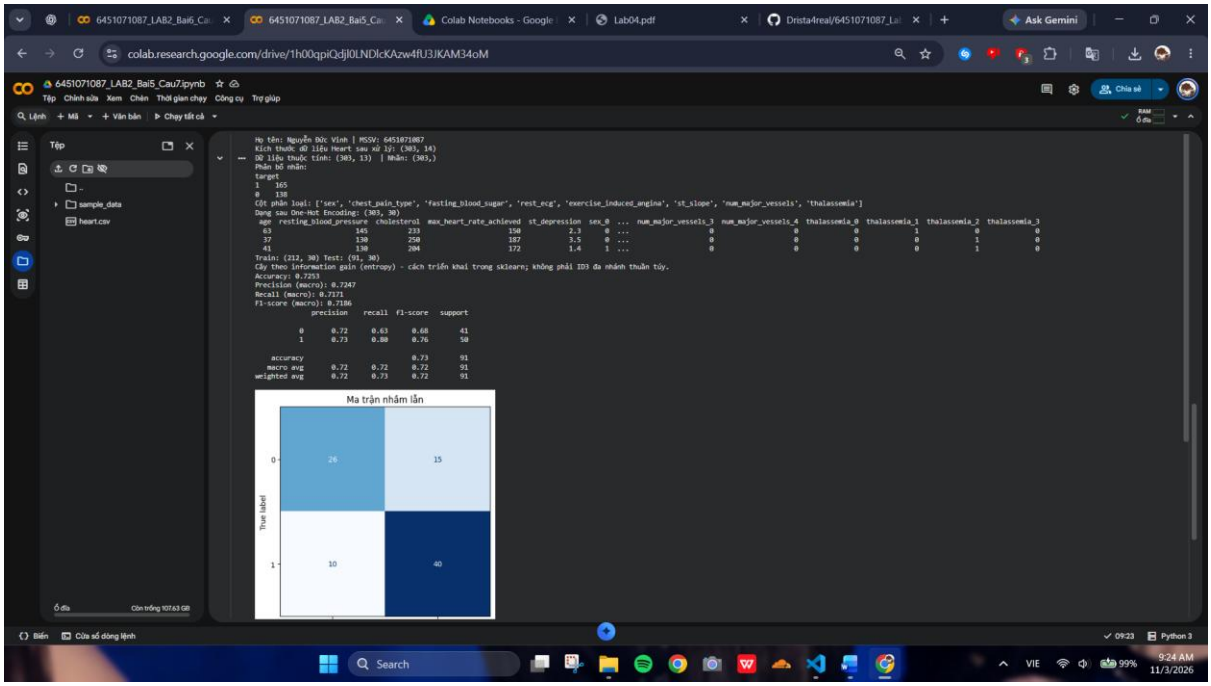
Câu 7: Cây entropy

Notebook: 6451071087_LAB2_Bai5_Cau7.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây theo information gain (entropy) - cách triển khai trong sklearn; không phải ID3 đa nhánh thuần
túy.')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (212, 30) Test: (91, 30)
Accuracy: 0.7253
Precision (macro): 0.7247
Recall (macro): 0.7171
F1-score (macro): 0.7186

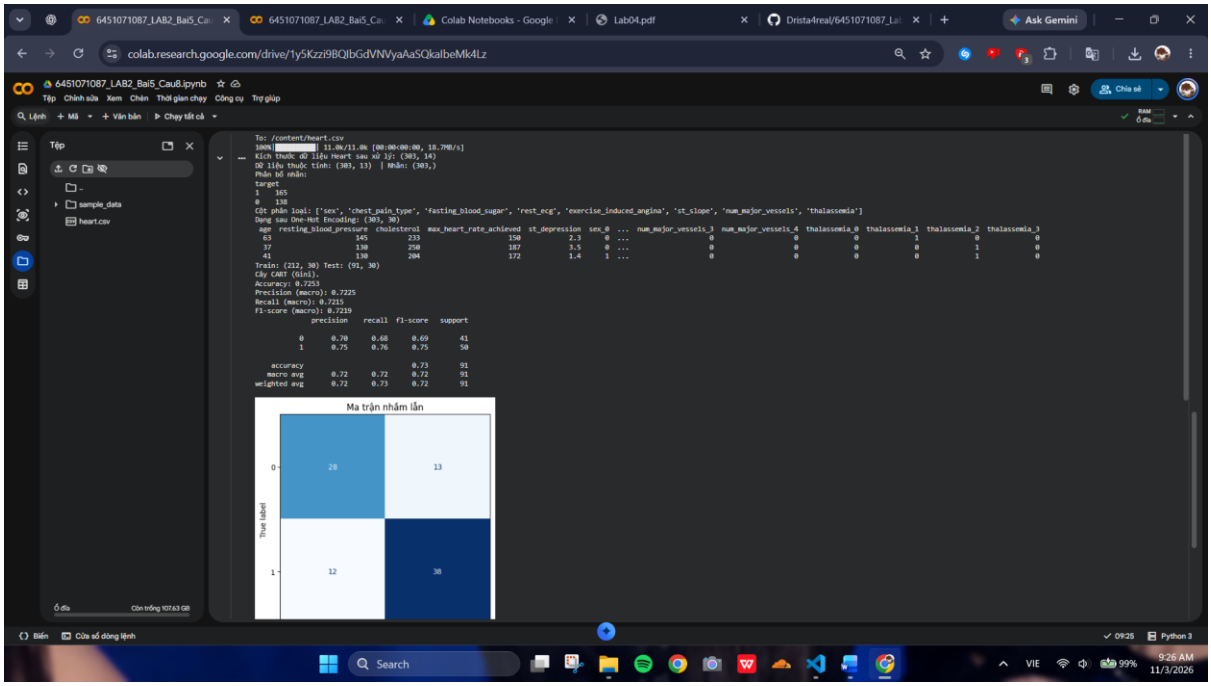
Câu 8: Cây Gini

Notebook: 6451071087_LAB2_Bai5_Cau8.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây CART (Gini).')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (212, 30) Test: (91, 30)
Accuracy: 0.7253
Precision (macro): 0.7225
Recall (macro): 0.7215
F1-score (macro): 0.7219

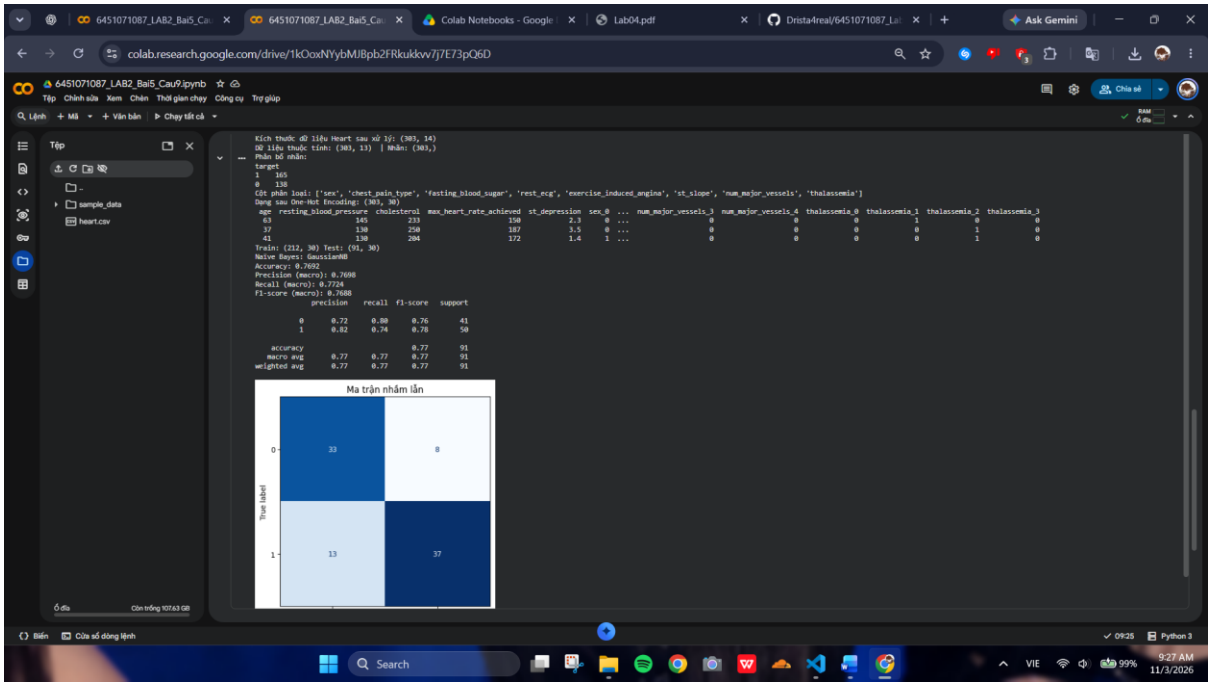
Câu 9: Naive Bayes

Notebook: 6451071087_LAB2_Bai5_Cau9.ipynb

Mã nguồn chính:

```
model = MultinomialNB(alpha=1.0) if DATASET == 'mush' else GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Naïve Bayes:', type(model).__name__)
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (212, 30) Test: (91, 30)
Naïve Bayes: GaussianNB
Accuracy: 0.7692
Precision (macro): 0.7698
Recall (macro): 0.7724
F1-score (macro): 0.7688

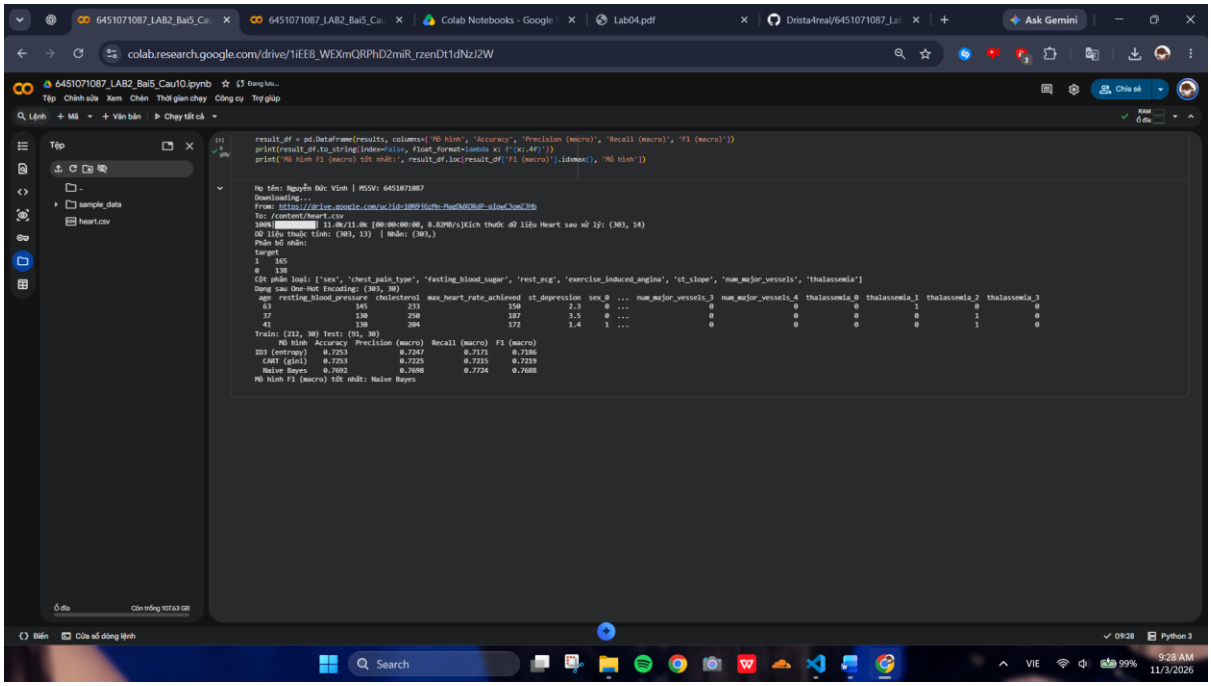
Câu 10: So sánh ID3, CART, NB

Notebook: 6451071087_LAB2_Bai5_Cau10.ipynb

Mã nguồn chính:

```
results = []
for title, m in [
    ('ID3 (entropy)', DecisionTreeClassifier(criterion='entropy', random_state=42)),
    ('CART (gini)', DecisionTreeClassifier(criterion='gini', random_state=42)),
    ('Naive Bayes', MultinomialNB(alpha=1) if DATASET == 'mush' else GaussianNB())
]:
    m.fit(X_train, y_train)
    yhat = m.predict(X_test)
    results.append([title, accuracy_score(y_test, yhat),
                    precision_score(y_test, yhat, average='macro', zero_division=0),
                    recall_score(y_test, yhat, average='macro', zero_division=0),
                    f1_score(y_test, yhat, average='macro', zero_division=0)])
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (212, 30) Test: (91, 30)

Mô hình	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
ID3 (entropy)	0.7253	0.7247	0.7171	0.7186
CART (gini)	0.7253	0.7225	0.7215	0.7219
Naive Bayes	0.7692	0.7698	0.7724	0.7688

Mô hình F1 (macro) tốt nhất: Naive Bayes

Nhận xét: Ưu tiên so sánh F1-macro khi dữ liệu có nhiều lớp hoặc các lớp mất cân bằng. Accuracy và Precision/Recall được ghi cụ thể phía trên; kết quả đánh giá chỉ phản ánh cách chia tập dữ liệu đã chọn.

Bài 6 (Trang 27) – Mushroom

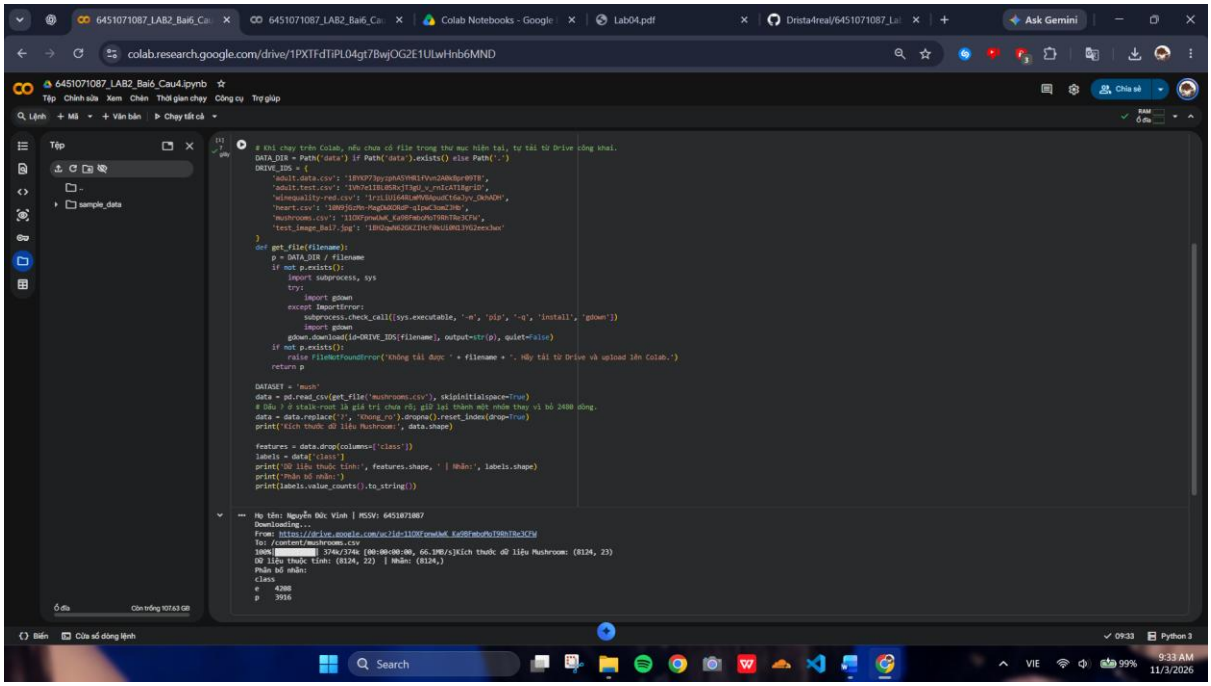
Câu 4: Tách class và thuộc tính

Notebook: 6451071087_LAB2_Bai6_Cau4.ipynb

Mã nguồn chính:

```
features = data.drop(columns=['class'])
labels = data['class']
print('Dữ liệu thuộc tính:', features.shape, ' | Nhãn:', labels.shape)
print('Phân bố nhãn:')
print(labels.value_counts().to_string())
```

Kết quả thực tế:



Kích thước dữ liệu Mushroom: (8124, 23)
Dữ liệu thuộc tính: (8124, 22) | Nhãn: (8124,)

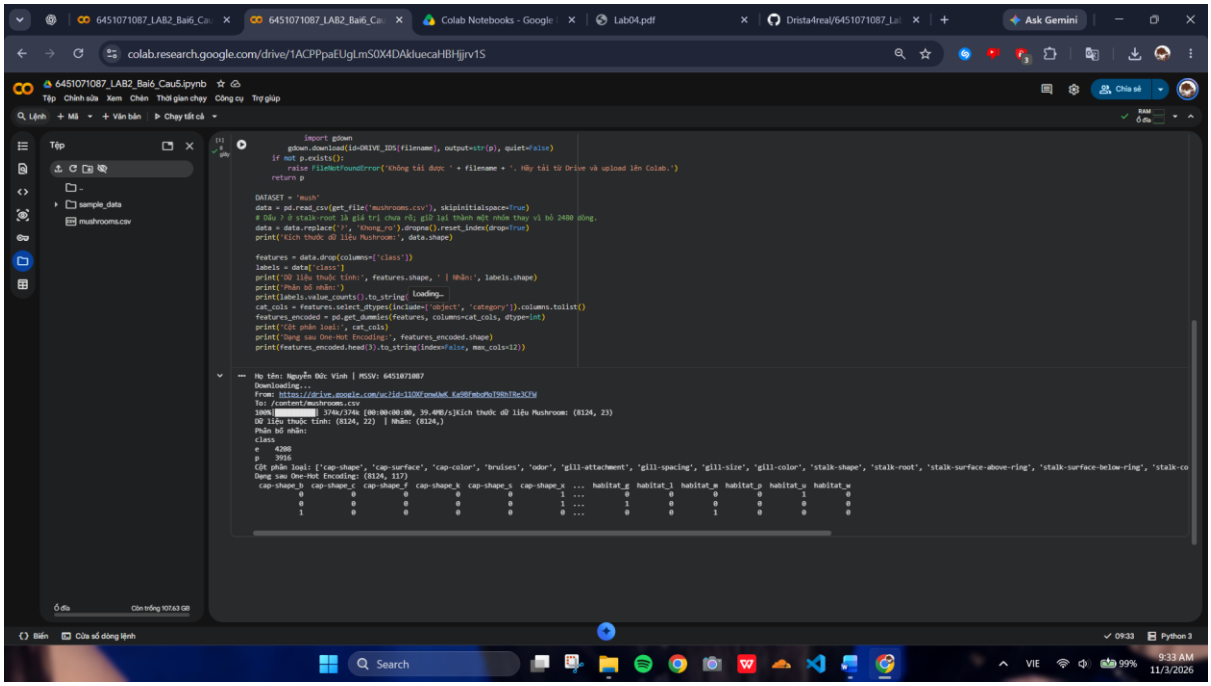
Câu 5: One-hot cho đặc điểm nấm

Notebook: 6451071087_LAB2_Bai6_Cau5.ipynb

Mã nguồn chính:

```
cat_cols = features.select_dtypes(include=['object', 'category']).columns.tolist()
features_encoded = pd.get_dummies(features, columns=cat_cols, dtype=int)
print('Cột phân loại:', cat_cols)
print('Dạng sau One-Hot Encoding:', features_encoded.shape)
print(features_encoded.head(3).to_string(index=False, max_cols=12))
```

Kết quả thực tế:



Kích thước dữ liệu Mushroom: (8124, 23)
Dữ liệu thuộc tính: (8124, 22) | Nhãn: (8124,)
Cột phân loại: ['cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor', 'gill-attachment', 'gill-spacing', 'gill-size', 'gill-
Dạng sau One-Hot Encoding: (8124, 117)

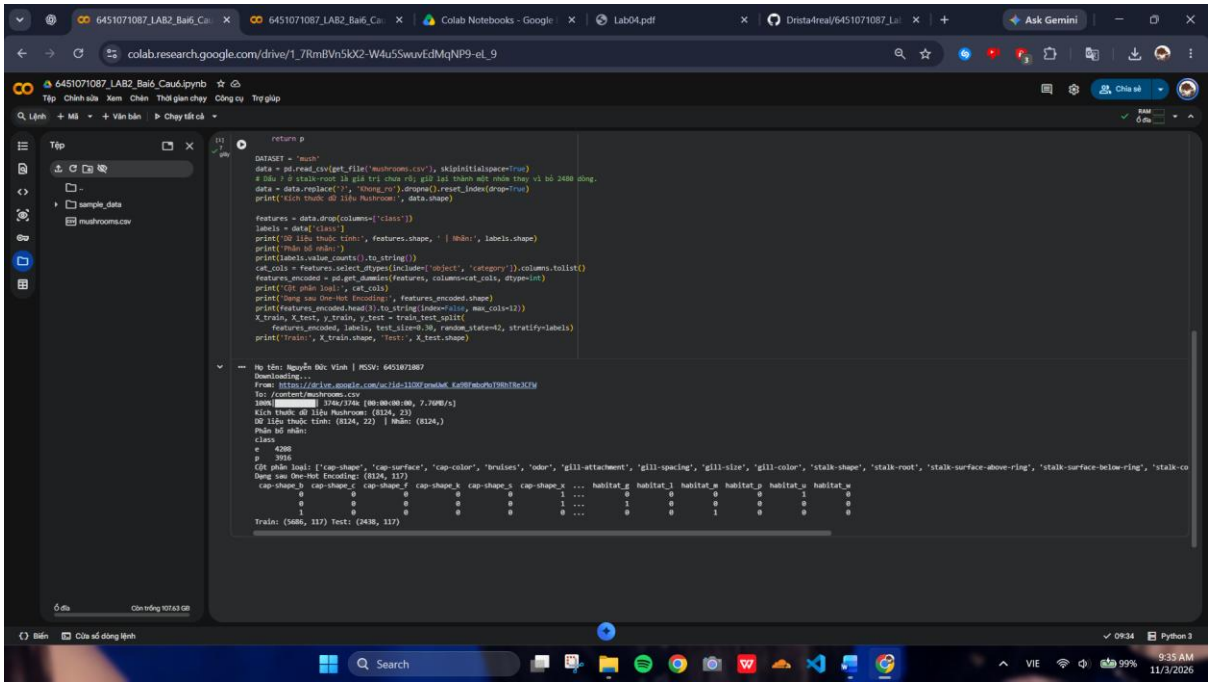
Câu 6: Chia train/test 70/30

Notebook: 6451071087_LAB2_Bai6_Cau6.ipynb

Mã nguồn chính:

```
X_train, X_test, y_train, y_test = train_test_split(
    features_encoded, labels, test_size=0.30, random_state=42, stratify=labels)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

Kết quả thực tế:



Kích thước dữ liệu Mushroom: (8124, 23)
Dữ liệu thuộc tính: (8124, 22) | Nhãn: (8124,)
Cột phân loại: ['cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor', 'gill-attachment', 'gill-spacing', 'gill-size', 'gill-
Dạng sau One-Hot Encoding: (8124, 117)
Train: (5686, 117) Test: (2438, 117)

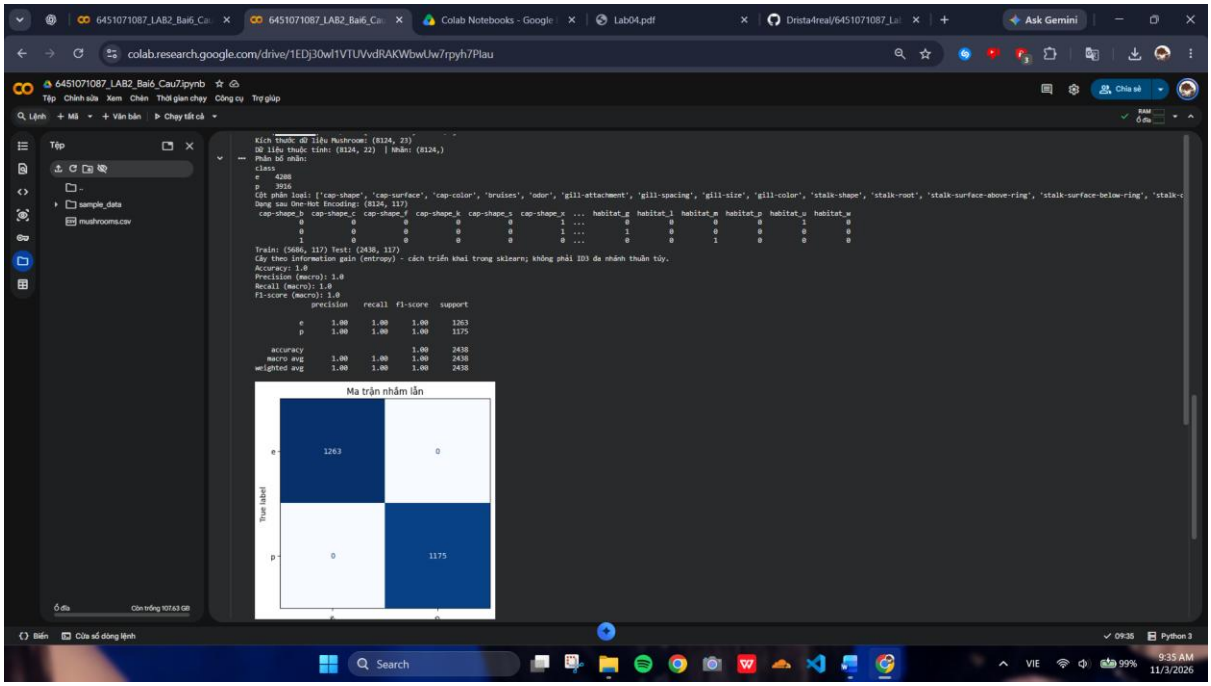
Câu 7: Cây entropy

Notebook: 6451071087_LAB2_Bai6_Cau7.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây theo information gain (entropy) - cách triển khai trong sklearn; không phải ID3 đa nhánh thuần
túy.')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (5686, 117) Test: (2438, 117)
Accuracy: 1.0
Precision (macro): 1.0
Recall (macro): 1.0
F1-score (macro): 1.0

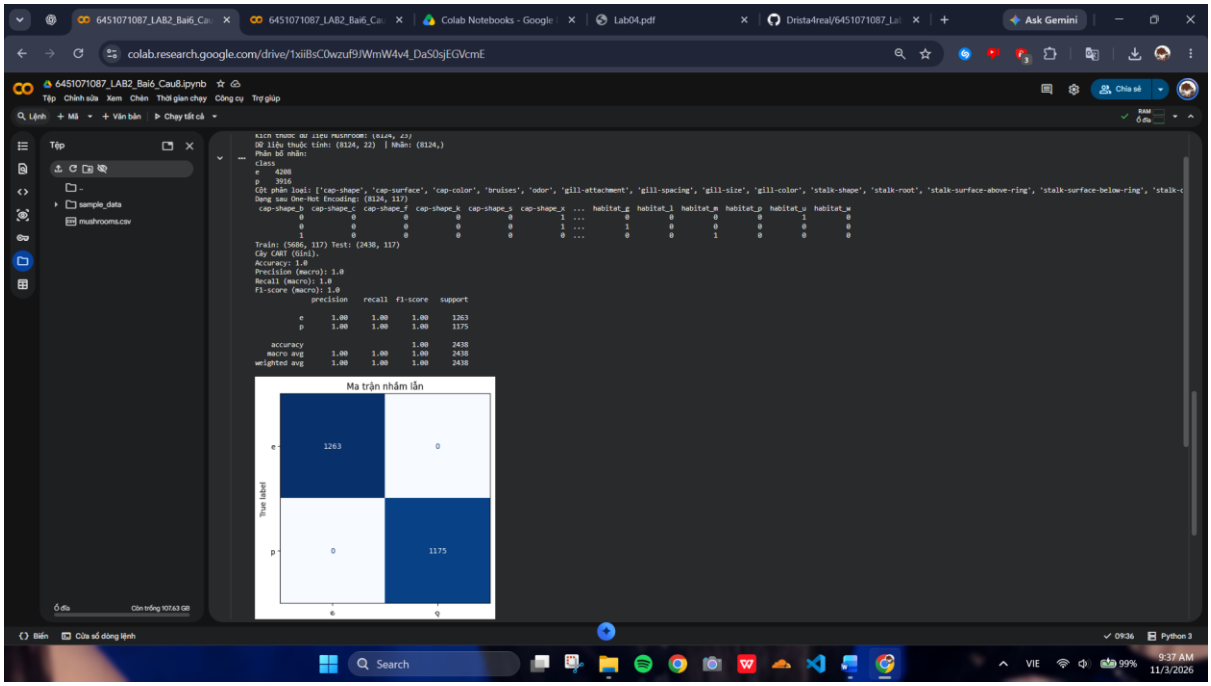
Câu 8: Cây Gini

Notebook: 6451071087_LAB2_Bai6_Cau8.ipynb

Mã nguồn chính:

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Cây CART (Gini).')
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (5686, 117) Test: (2438, 117)
Accuracy: 1.0
Precision (macro): 1.0
Recall (macro): 1.0
F1-score (macro): 1.0

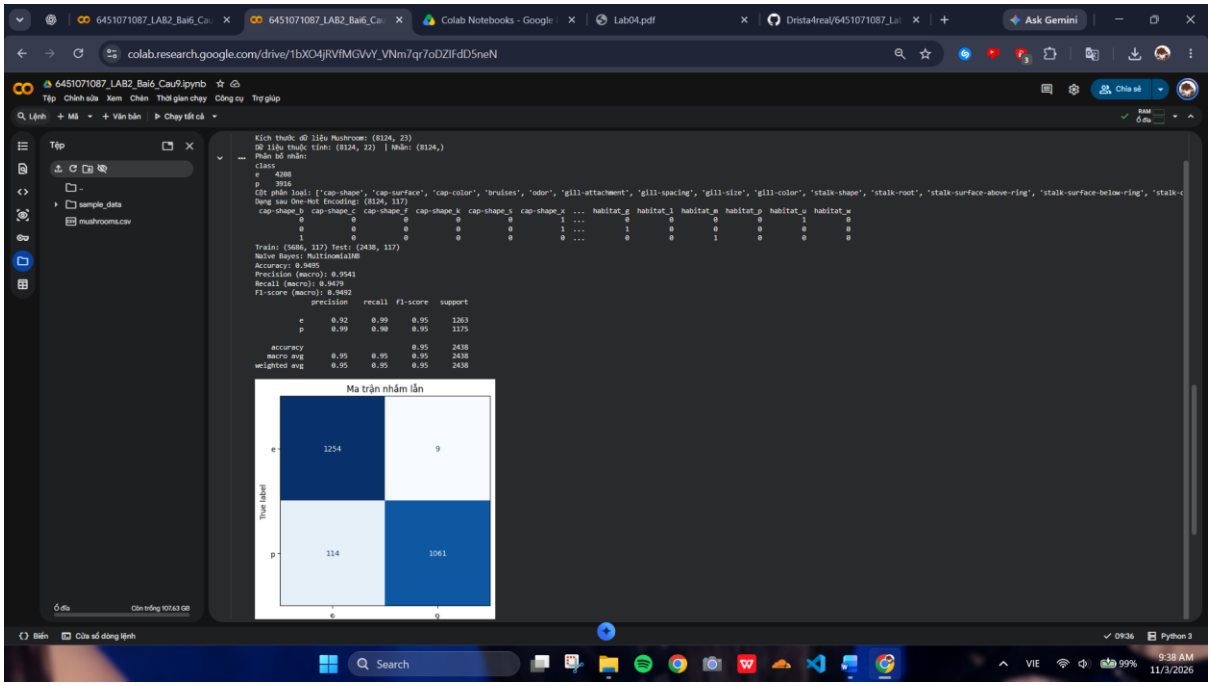
Câu 9: Naive Bayes

Notebook: 6451071087_LAB2_Bai6_Cau9.ipynb

Mã nguồn chính:

```
model = MultinomialNB(alpha=1.0) if DATASET == 'mush' else GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Naïve Bayes:', type(model).__name__)
print('Accuracy:', round(accuracy_score(y_test, pred), 4))
print('Precision (macro):', round(precision_score(y_test, pred, average='macro', zero_division=0), 4))
print('Recall (macro):', round(recall_score(y_test, pred, average='macro', zero_division=0), 4))
print('F1-score (macro):', round(f1_score(y_test, pred, average='macro', zero_division=0), 4))
print(classification_report(y_test, pred, zero_division=0))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, cmap='Blues', colorbar=False,
xticks_rotation=40)
plt.title('Ma trận nhầm lẫn')
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Train: (5686, 117) Test: (2438, 117)
Naïve Bayes: MultinomialNB
Accuracy: 0.9495
Precision (macro): 0.9541
Recall (macro): 0.9479
F1-score (macro): 0.9492

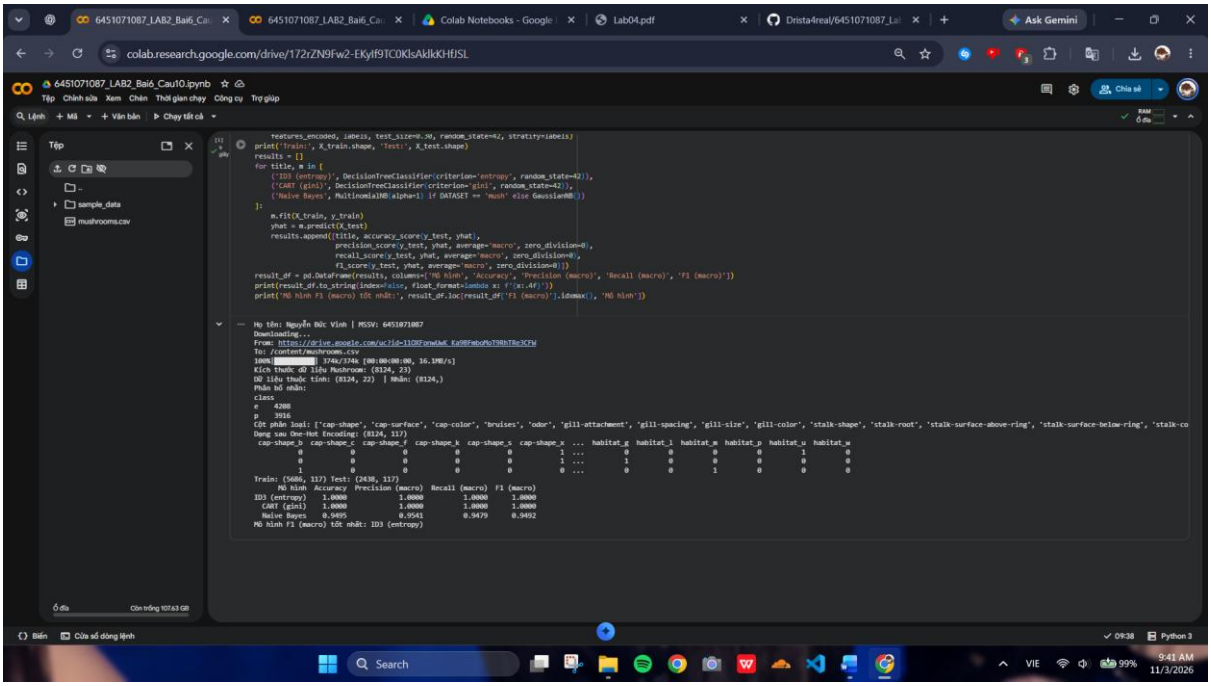
Câu 10: So sánh ID3, CART, NB

Notebook: 6451071087_LAB2_Bai6_Cau10.ipynb

Mã nguồn chính:

```
results = []
for title, m in [
    ('ID3 (entropy)', DecisionTreeClassifier(criterion='entropy', random_state=42)),
    ('CART (gini)', DecisionTreeClassifier(criterion='gini', random_state=42)),
    ('Naive Bayes', MultinomialNB(alpha=1) if DATASET == 'mush' else GaussianNB())
]:
    m.fit(X_train, y_train)
    yhat = m.predict(X_test)
    results.append([title, accuracy_score(y_test, yhat),
                    precision_score(y_test, yhat, average='macro', zero_division=0),
                    recall_score(y_test, yhat, average='macro', zero_division=0),
                    f1_score(y_test, yhat, average='macro', zero_division=0)])
# ... (mã đầy đủ trong file .ipynb tương ứng)
```

Kết quả thực tế:



Mô hình	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
ID3 (entropy)	1.0000	1.0000	1.0000	1.0000
CART (gini)	1.0000	1.0000	1.0000	1.0000
Naive Bayes	0.9495	0.9541	0.9479	0.9492

Mô hình F1 (macro) tốt nhất: ID3 (entropy)

Nhận xét: Ưu tiên so sánh F1-macro khi dữ liệu có nhiều lớp hoặc các lớp mất cân bằng. Accuracy và Precision/Recall được ghi cụ thể phía trên; kết quả đánh giá chỉ phản ánh cách chia tập dữ liệu đã chọn.

Bài 7 (Trang 27 - 31) – Nhận dạng chữ số

Nhận dạng chữ số viết tay bằng sklearn digits 8x8

Notebook: 6451071087_LAB2_Bai7.ipynb

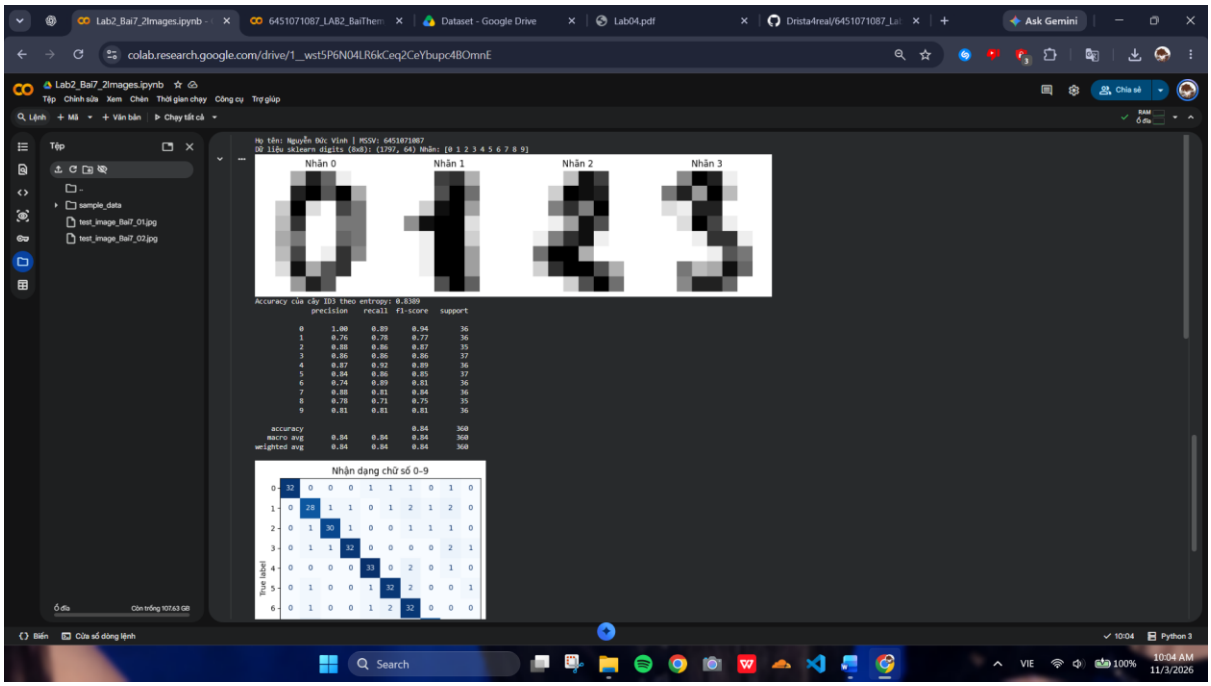
Mã nguồn chính:

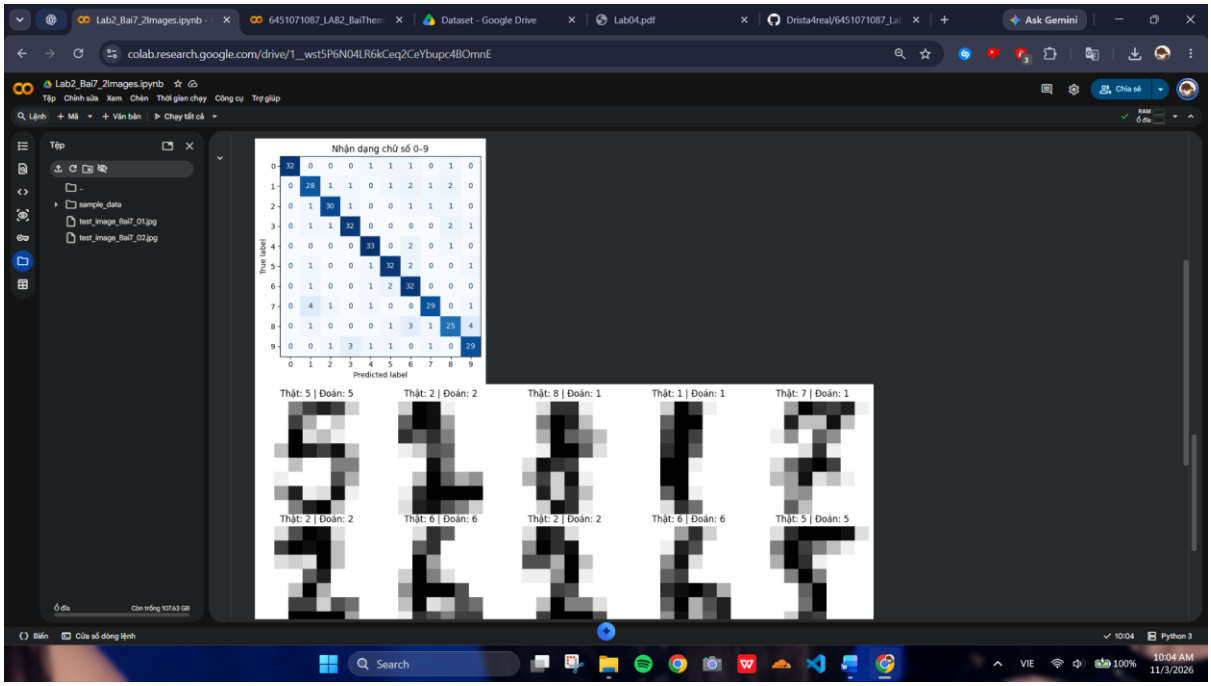
```
DIGITS = load_digits()
X = DIGITS.data                  # 1797 ảnh 8x8, mỗi ảnh 64 đặc trưng
y = DIGITS.target
print('Dữ liệu sklearn digits (8x8):', X.shape, 'Nhãn:', np.unique(y))
fig, axes = plt.subplots(1, 4, figsize=(10, 3))
for i, ax in enumerate(axes):
    j = np.where(y==i)[0][0]
    ax.imshow(DIGITS.images[j], cmap='gray_r'); ax.set_title(f'Nhãn {i}'); ax.axis('off')
plt.tight_layout(); plt.show()
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42,stratify=y)
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train,y_train)
pred=model.predict(X_test)
print('Accuracy của cây ID3 theo entropy:',round(accuracy_score(y_test,pred),4))
# ... huấn luyện cây entropy, đánh giá ma trận nhầm lẫn,
# ... dự đoán cả hai ảnh bên ngoài theo vòng lặp bên dưới.
# Bài 7 đã xử lý CẢ 2 ảnh, không chỉ một ảnh:
for filename, nhan in [("test_image_Bai7_01.jpg", 2), ("test_image_Bai7_02.jpg", 1)]:
```

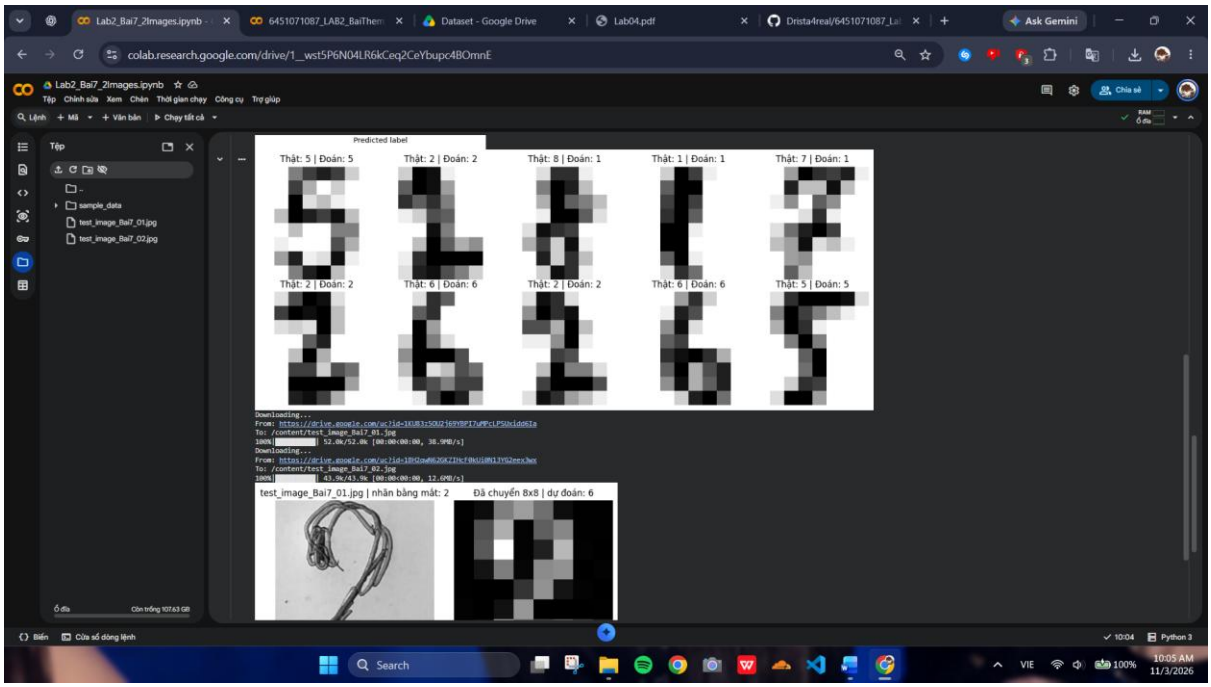
```
anh_goc, vector8x8 = preprocess_handwritten_image(get_file(filename))
du_doan = model.predict(vector8x8.reshape(1, 64))[0]
print(filename, "Nhãn:", nhan, "Dự đoán:", du_doan)
```

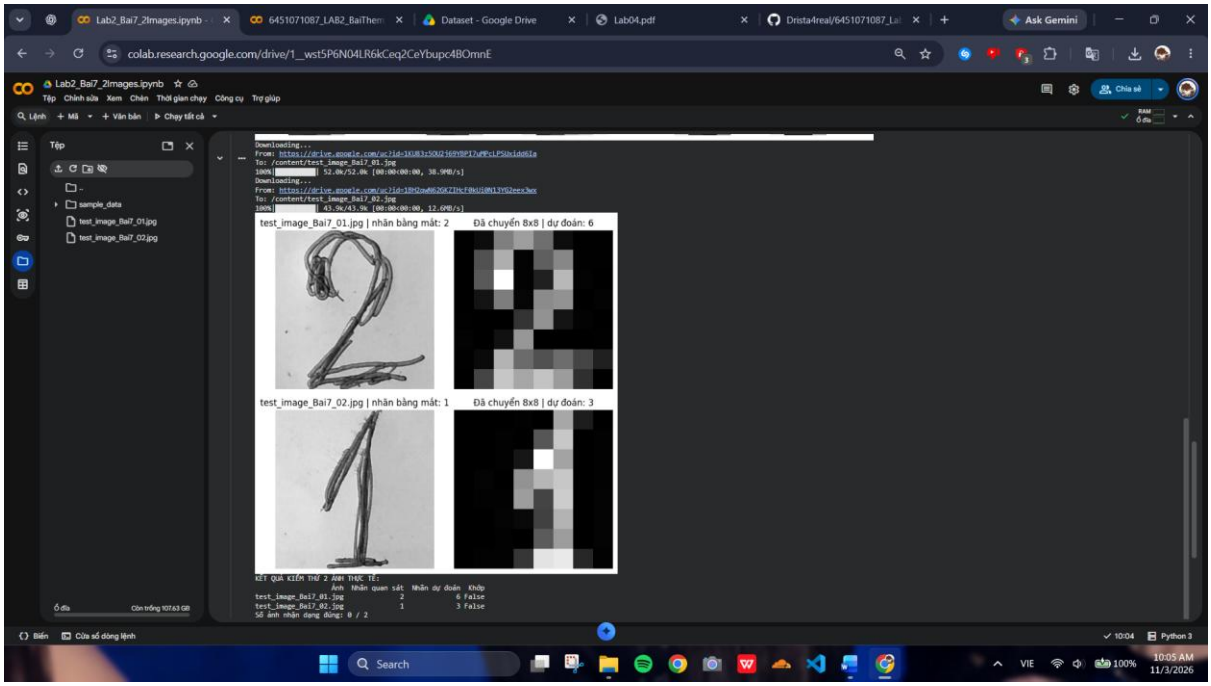
Kết quả thực tế:

Dữ liệu sklearn digits: 1.797 ảnh, mỗi ảnh 8×8; độ chính xác trên tập kiểm thử (360 mẫu): 83,89%.
Ảnh kiểm thử số 1 (chữ số 2): mô hình dự đoán 6 — sai.
Ảnh kiểm thử số 2 (chữ số 1): mô hình dự đoán 3 — sai.
Kết quả trên 2 ảnh chụp thực tế: 0/2 ảnh nhận dạng đúng. Ảnh chụp khác phân phối với dữ liệu huấn luyện 8×8; cần cải thiện tiền xử lý hoặc phương pháp huấn luyện để nâng khả năng tổng quát hóa.




Nhận dạng chữ số 0–9
True label
Predicted label
Thật: 5 | Đoán: 5
Thật: 2 | Đoán: 2
Thật: 8 | Đoán: 1
Thật: 1 | Đoán: 1
Thật: 7 | Đoán: 1
Thật: 2 | Đoán: 2
Thật: 6 | Đoán: 6
Thật: 2 | Đoán: 2
Thật: 6 | Đoán: 6
Thật: 5 | Đoán: 5


Predicted label
Thật: 5 | Đoán: 5
Thật: 2 | Đoán: 2
Thật: 8 | Đoán: 1
Thật: 1 | Đoán: 1
Thật: 7 | Đoán: 1
Thật: 2 | Đoán: 2
Thật: 6 | Đoán: 6
Thật: 2 | Đoán: 2
Thật: 6 | Đoán: 6
Thật: 5 | Đoán: 5
test_image_Bai7_01.jpg | nhãn bằng mắt: 2
Đã chuyển 8x8 | dự đoán: 6



Bài 2 (Trang 32)

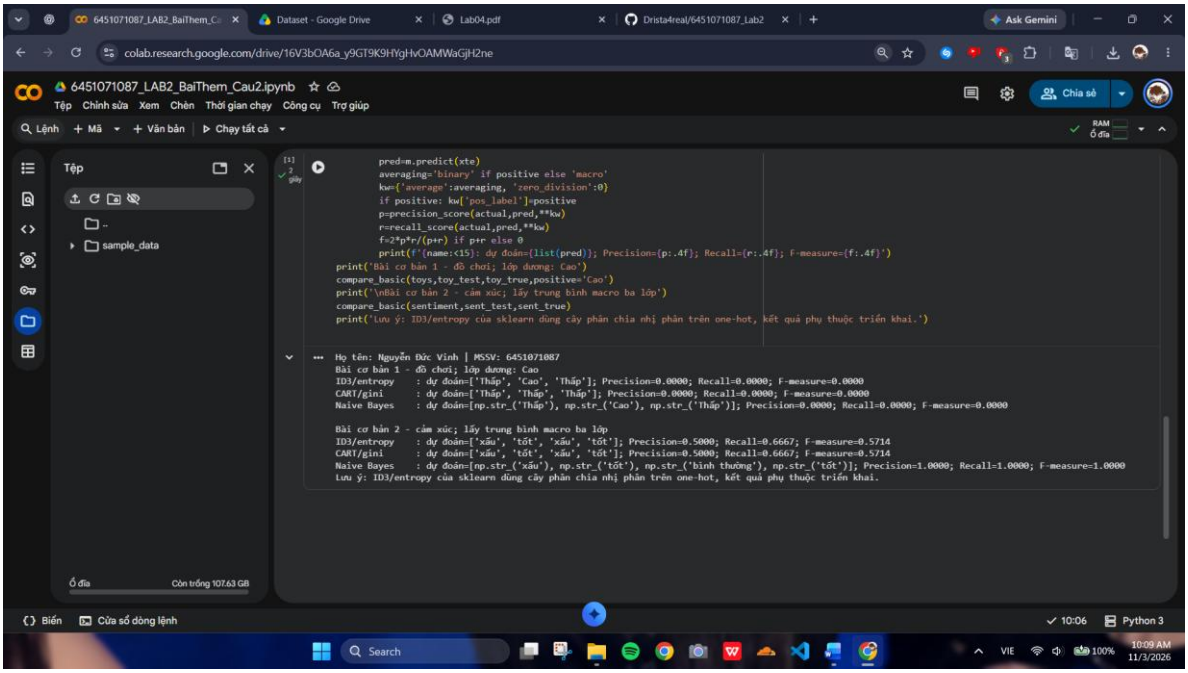
F-measure bài tập cơ bản

Notebook: 6451071087_LAB2_BaiThem_Cau2.ipynb

Mã nguồn chính:

```
F = 2 * Precision * Recall / (Precision + Recall)
# Với mỗi bài cơ bản: train ID3/CART/Naive Bayes, dự đoán 3 hoặc 4 mẫu, tính Precision và Recall.
compare_basic(toys, toy_test, toy_true, positive="Cao")
compare_basic(sentiment, sent_test, sent_true)
```

Kết quả thực tế:



Bài cơ bản 1 - đồ chơi; lớp dương: Cao

ID3/entropy : dự đoán=['Thấp', 'Cao', 'Thấp']; Precision=0.0000; Recall=0.0000; F-measure=0.0000
CART/gini : dự đoán=['Thấp', 'Thấp', 'Thấp']; Precision=0.0000; Recall=0.0000; F-measure=0.0000
Naive Bayes : dự đoán=[np.str_('Thấp'), np.str_('Cao'), np.str_('Thấp')]; Precision=0.0000; Recall=0.0000; F-measure=0.0000

Bài cơ bản 2 - cảm xúc; lấy trung bình macro ba lớp
ID3/entropy : dự đoán=['xấu', 'tốt', 'xấu', 'tốt']; Precision=0.5000; Recall=0.6667; F-measure=0.5714
CART/gini : dự đoán=['xấu', 'tốt', 'xấu', 'tốt']; Precision=0.5000; Recall=0.6667; F-measure=0.5714
Naive Bayes : dự đoán=[np.str_('xấu'), np.str_('tốt'), np.str_('bình thường'), np.str_('tốt')]; Precision=1.0000; Recall=1.0000; F-measure=1.0000

--- HẾT---